Supervised Learning

Reinforcement Learning

Supervised Learning

- Function approximation as backbone of modern RL
- Introduction to supervised learning
- Deep learning: neural networks, convolutional neural networks

Approximate Solution Methods

online setting: learning while interacting with environment
offline setting: learning from limited data without further interaction with environment

state/action values as parametrized function (instead of table)

- variables/features describing different states
- parameters (e.g., weights in neural network) to be learned

objective function for supervised learning (e.g., squared error loss):

$$\mathcal{L}(\boldsymbol{w}) = \sum_{s} \left(v(s) - \hat{v}(s; \boldsymbol{w}) \right)^2$$

parameters/weights to be optimized

→ RL problem expressed in supervised learning setup

but $v(s)$ still calculated via RL methods (e.g., bootstrapping)

Idea of Supervised Learning

Target Quantity

- **known in training**: labeled samples or observations from past
- to be **predicted** for unknown cases (e.g., future values)

Features

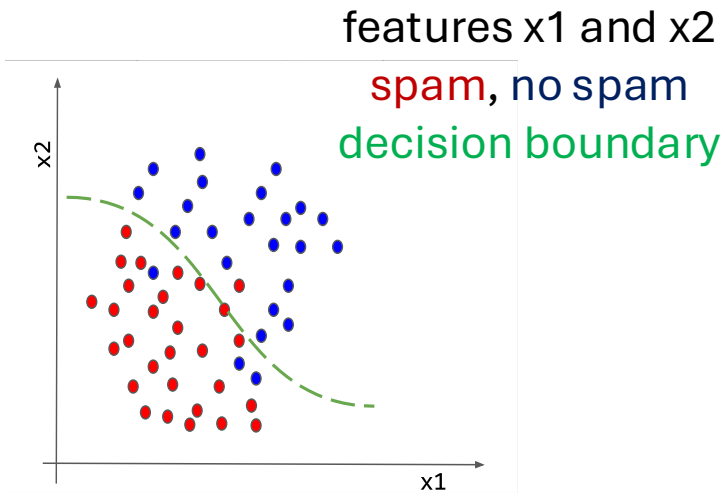
(prepared) input information that is
- correlated to target quantity
- known at prediction time

Example: Spam Filtering

classify emails as spam or no spam

*use accordingly **labeled emails as training set***

use information like occurrence of specific words or email length as **features**



Supervised Learning in Mathematical Terms

map input to output: $y = f(\boldsymbol{x})$

random variables Y and $\boldsymbol{X} = (X_1, X_2, \cdots, X_p)$ ← usually high-dimensional

training (e.g., curve fitting / parameter estimation):

fit data set of $(y_i, \boldsymbol{x}_i)$ pairs → minimize deviations between y and $f(\boldsymbol{x})$

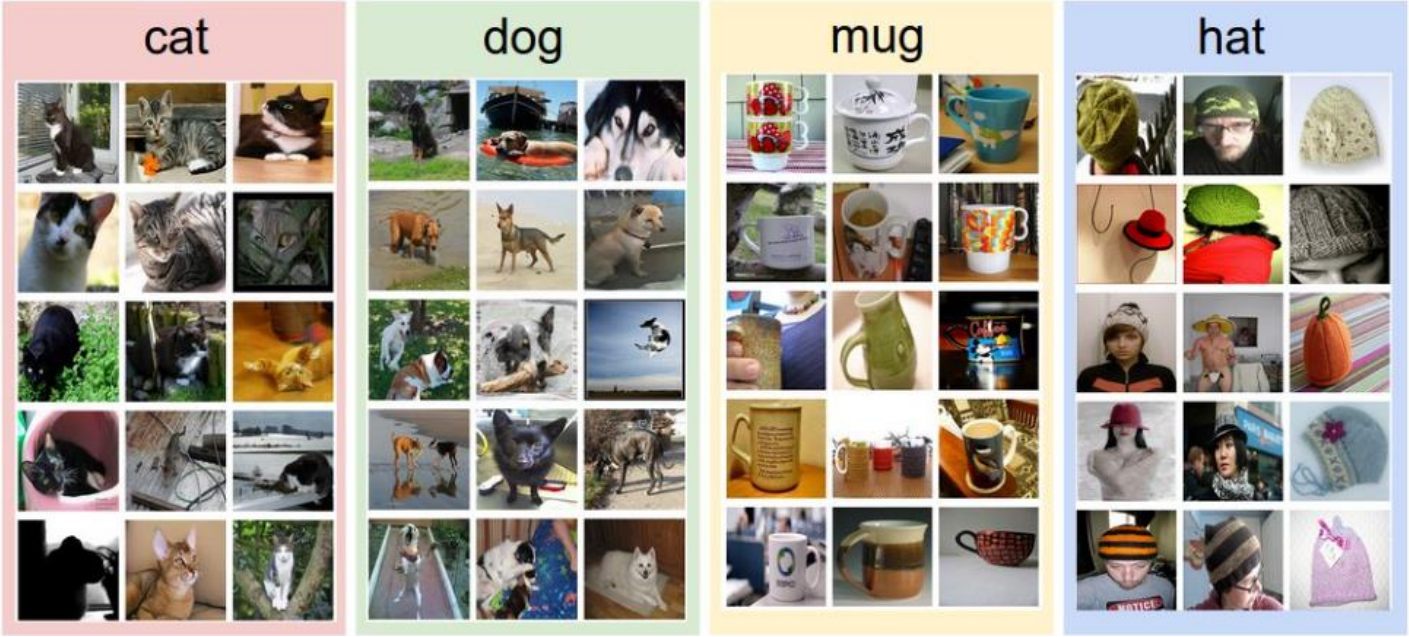
discriminative models: predict conditional density function $p(y|\boldsymbol{x})$

as opposed to generative models: predict $p(y, \boldsymbol{x})$ (or just $p(\boldsymbol{x})$) → $\boldsymbol{x}$ not given, more difficult

Example: Image Classification

training data set:

labels:



test data with learned classifier:

$f($ $) =$ "Cat"

$f($ $) =$ "Dog"

Discrete Intensity Levels

image data: 2D grid (matrix) of pixels with different intensity values

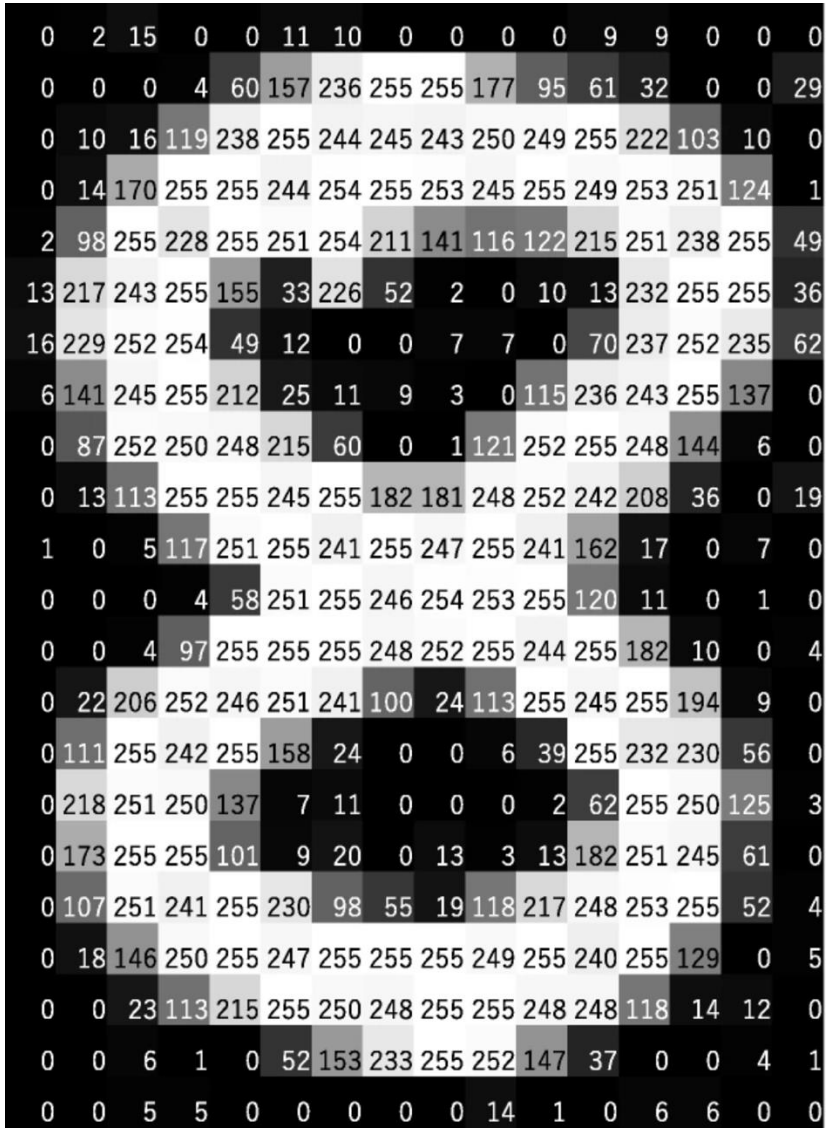
intensity resolution:

common to use one byte (2^8) to express possible intensity values (8-bit image)

→ 0 = black, 255 = white

storage requirement for, e.g., 1024×1024 8-bit image (grayscale): ~1 MB

($\times$ 3 for RGB color images)



0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

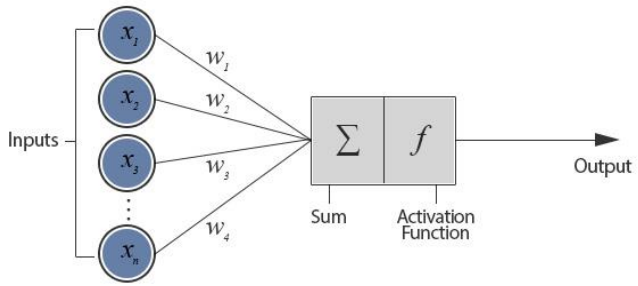
source

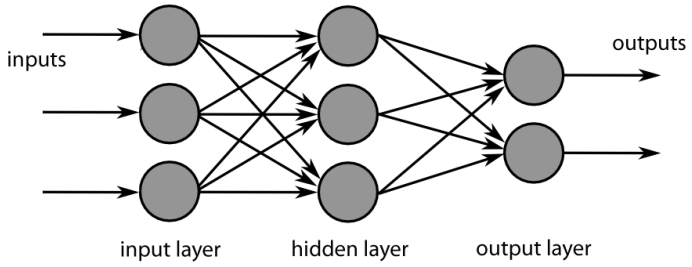
Algorithmic Families of Supervised Learning

parametric models

linear models

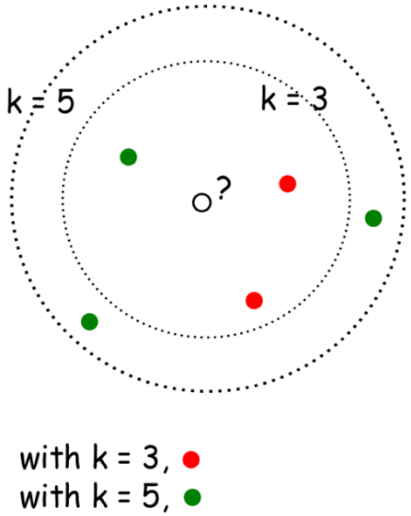
neural networks





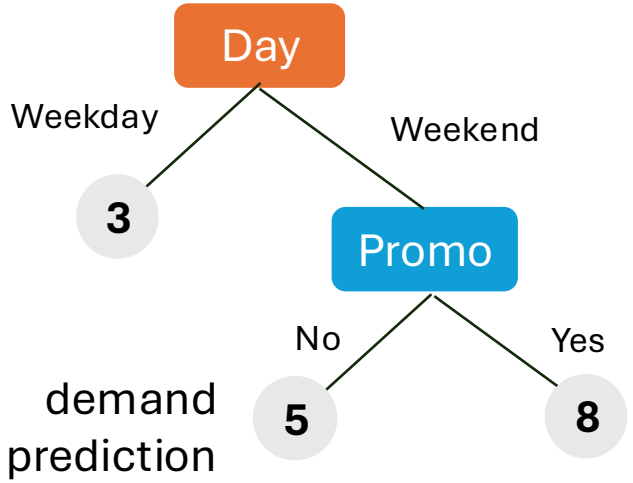
non-parametric models: similarity-based learning (analogies)

nearest-neighbor-style/local methods



kernel machines (e.g., support-vector machines)

decision trees: rule learning



often used in ensemble methods

- bagging: random forests
- boosting: gradient boosting

Linear Regression

fit:

$$y_i = \overbrace{b + \sum_{j=1}^{p} w_j \, x_{ij}}^{f(\boldsymbol{x}_i)} + \varepsilon_i$$

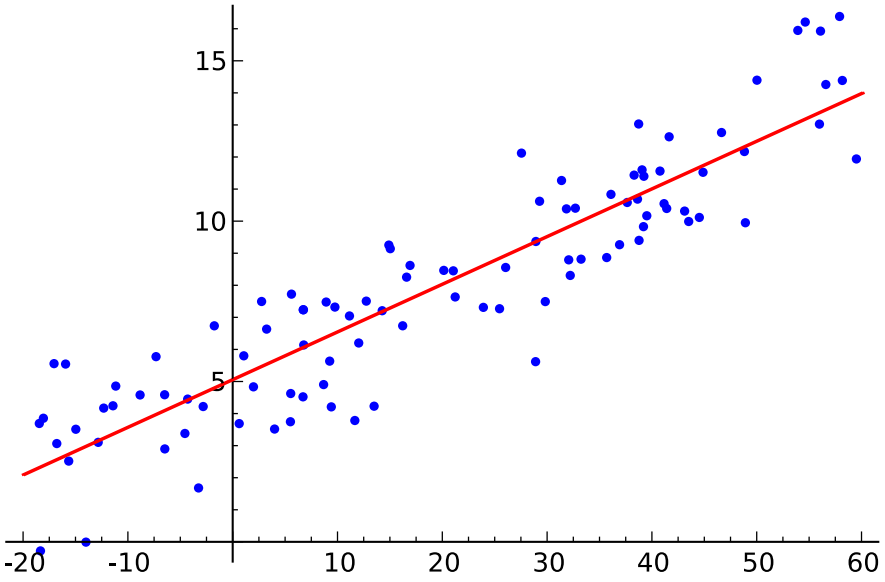
Gaussian

predict:

$$\hat{y}_i = E[Y|\boldsymbol{X} = \boldsymbol{x}_i] = f(\boldsymbol{x}_i) = b + \sum_{j=1}^{p} w_j \, x_{ij}$$

$$p(y|\boldsymbol{x}_i) = \mathcal{N}(y; \hat{y}_i, \sigma^2)$$

Gaussian mean variance

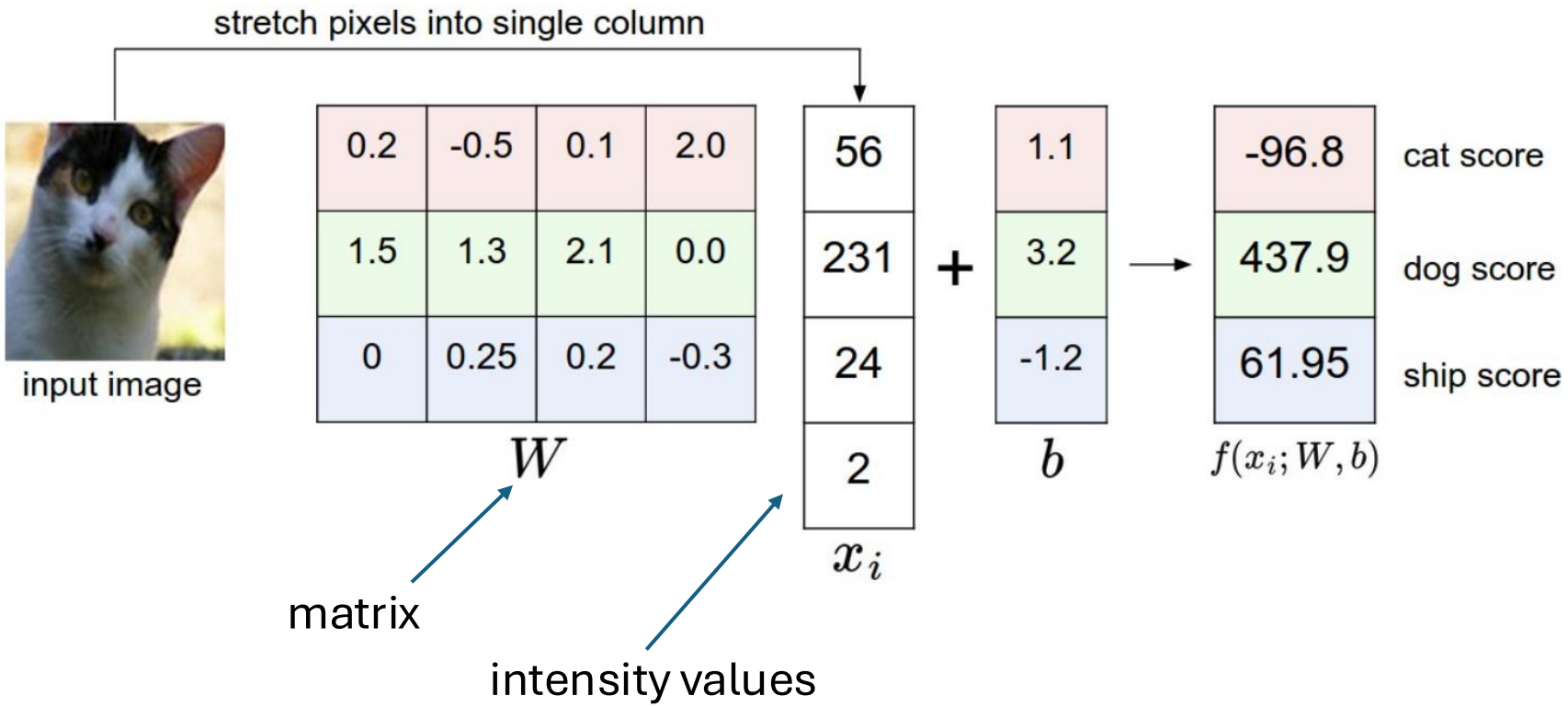


parameter estimation:
e.g., least squares

parameters to be estimated: $b, \boldsymbol{w}$
$\rightarrow \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - f(\boldsymbol{x}_i)\right)^2$

Image Classification with Linear Regression

simplified example: 4-pixel image, 3 classes



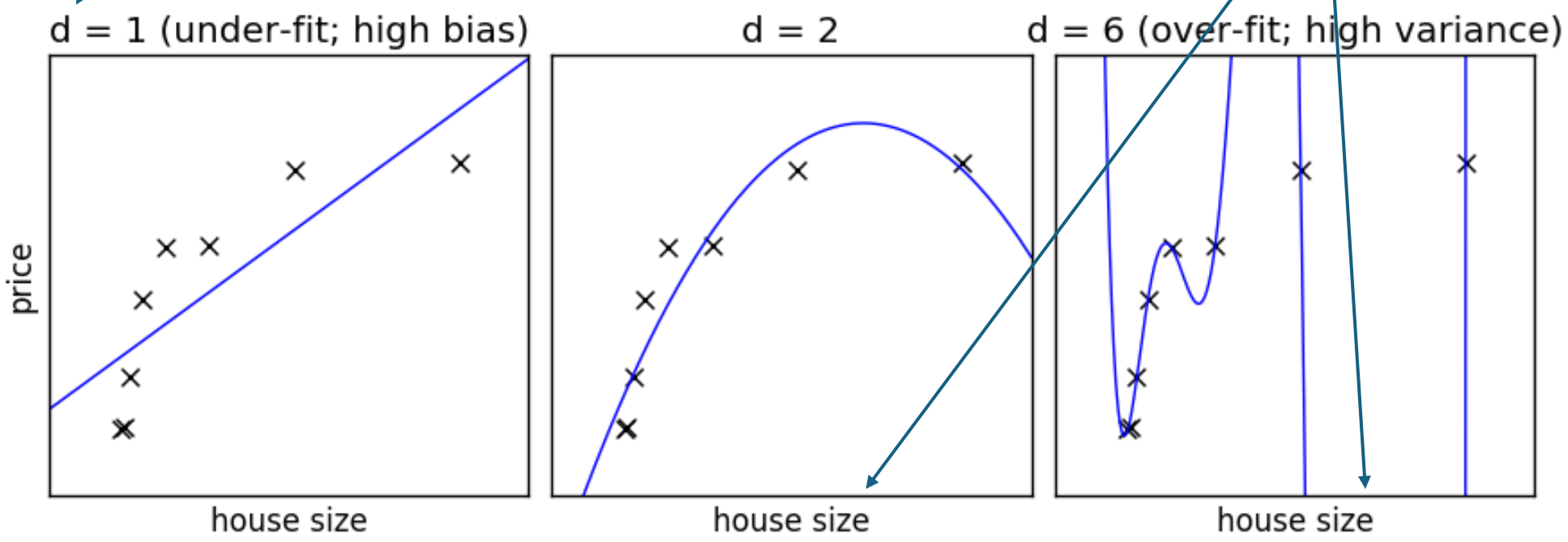
need for one common image size per model

Under- and Over-Fitting

example: polynomial fit

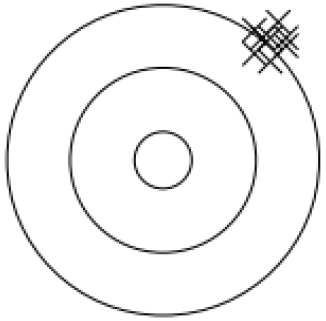
need to predict new samples (not in training data set): interpolation

degree of fitted polynomial

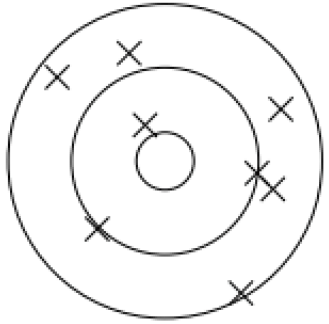


Bias, Variance, Irreducible Error

think of fitting ML algorithms as repeatable processes with different data sets



bias
due to too simplistic model
(same for all training data sets)
"underfitting"



variance
due to sensitivity to specifics (noise)
of different training data sets
"overfitting"

irreducible error (aka Bayes error):

inherent randomness (target generated from random variable)

→limiting accuracy of ideal model

Bias-Variance Tradeoff

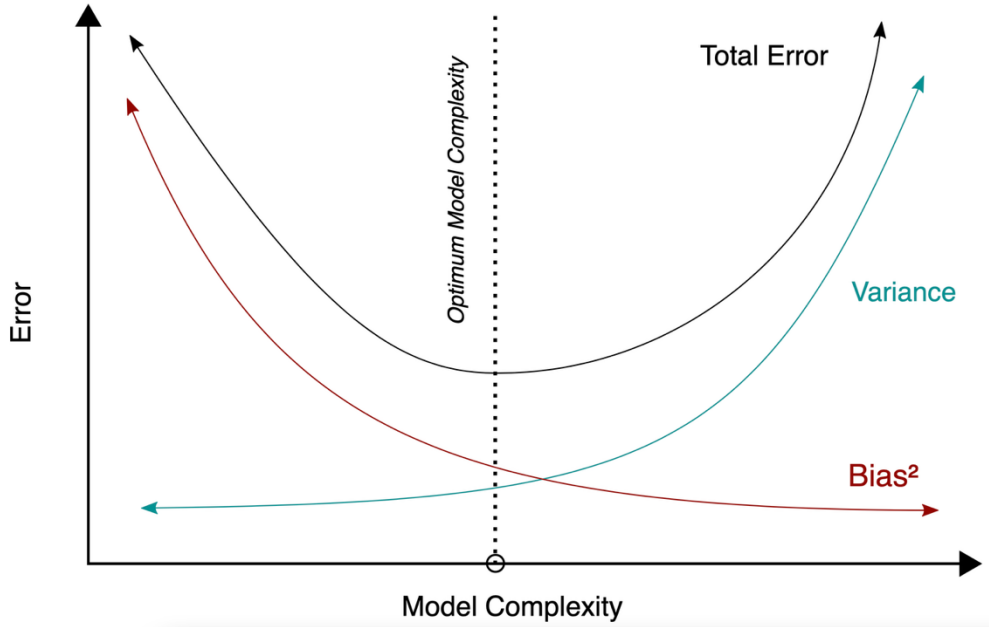
models of higher complexity have lower bias but higher variance

(given the same number of training examples)

generalization error follows U-shaped curve: overfitting once model complexity (number of parameters) passes certain threshold

overfitting: variance term dominating test error

→ increasing model complexity increases test error



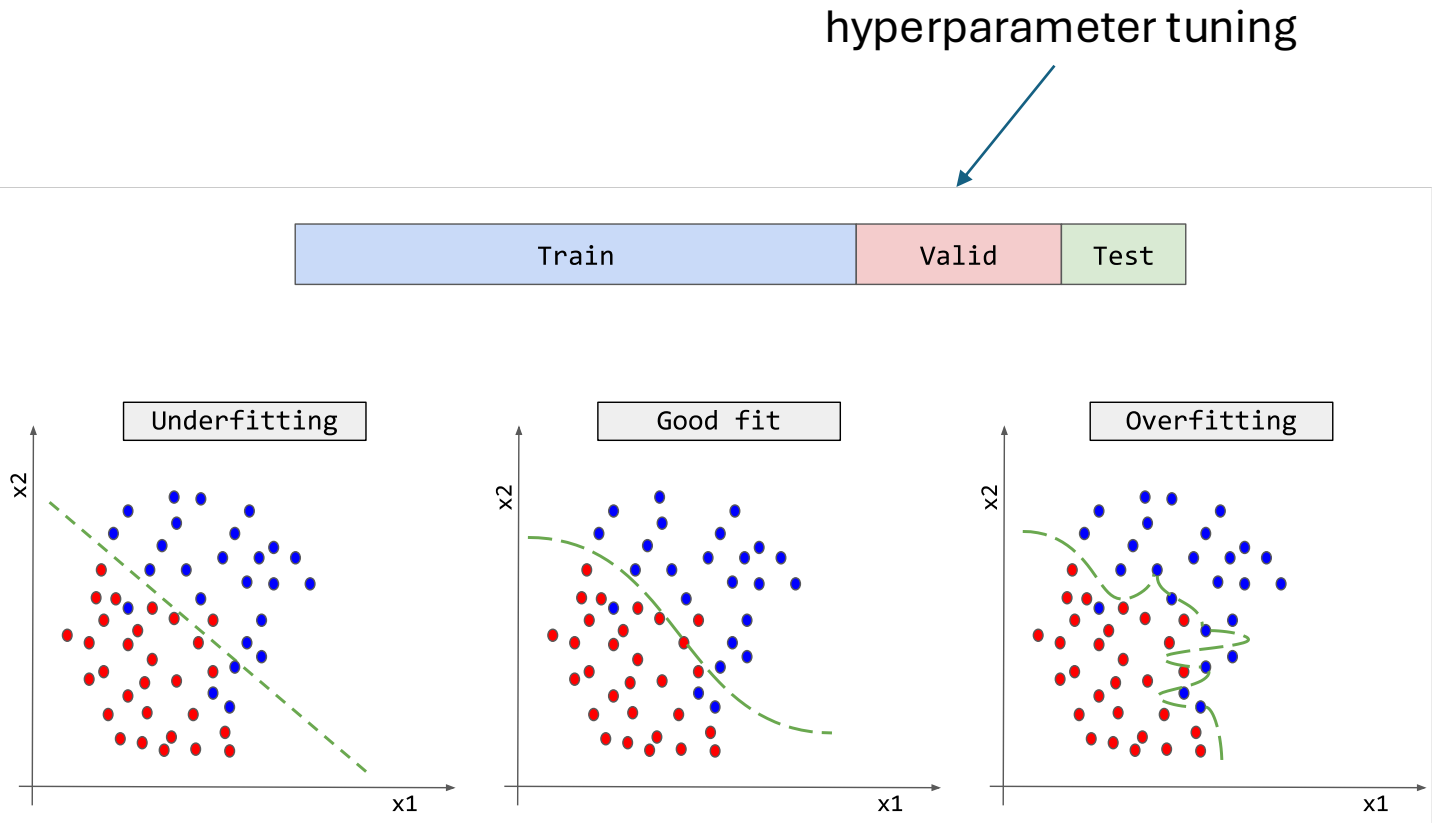
but beware: training error keeps decreasing with more complex model ...

Hyperparameters

model complexity often controlled via hyperparameters (parameters not fitted but set in advance)

examples:

- degree of fitted polynomial d
- number of considered nearest neighbors k
- maximum depth of decision tree
- number of hidden nodes in neural network layer



Train-Dev-Test vs. Model fitting

Generalization

core of ML: **empirical risk minimization** (training error) as proxy for minimizing unknown population risk (test error)

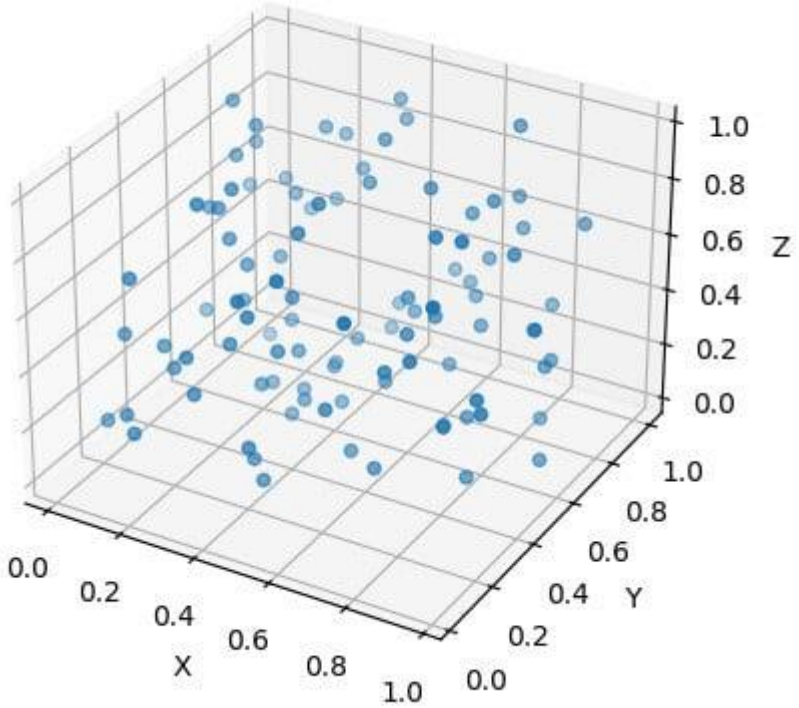
generalization gap: difference between test and training error

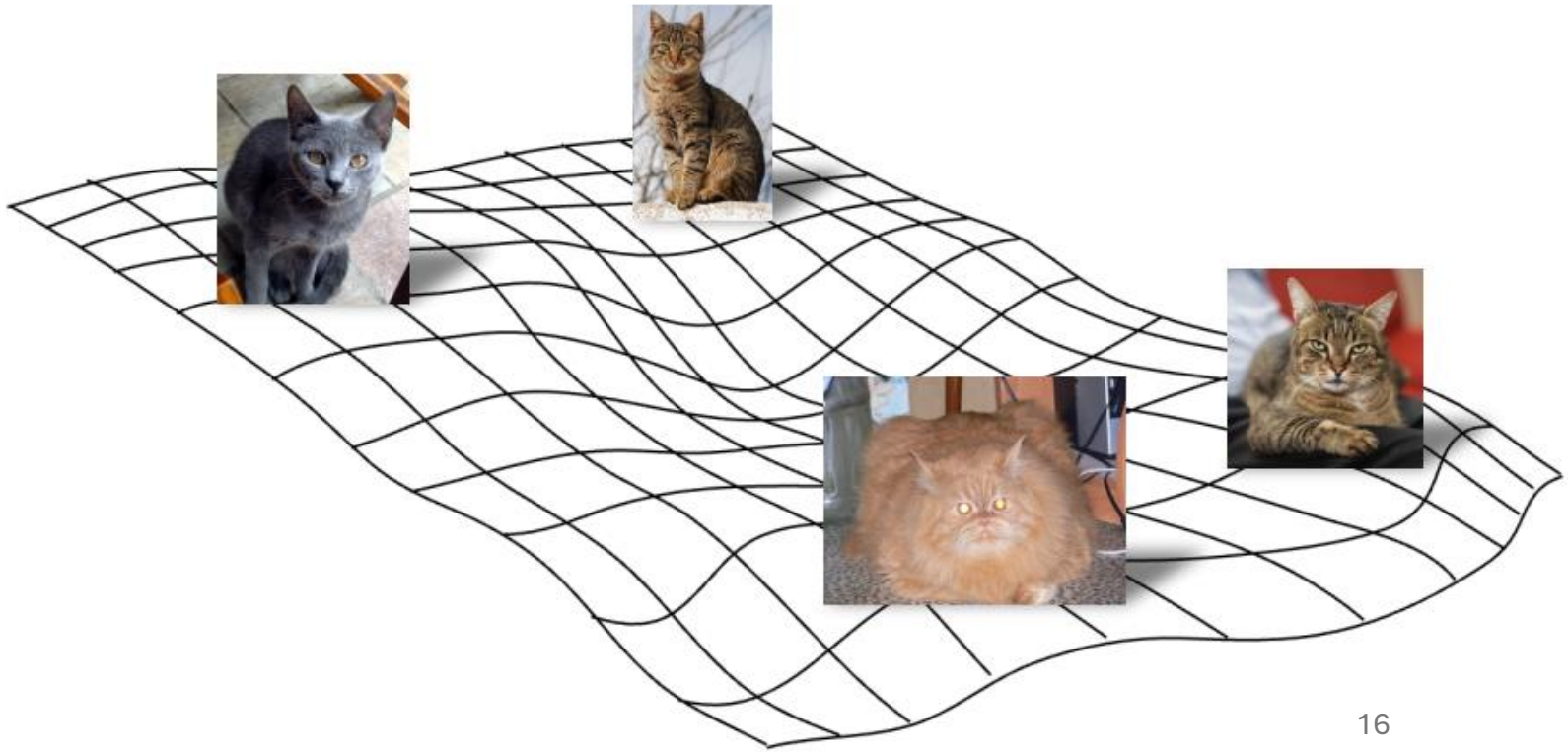
curse of dimensionality: typically, many features (dimensions)

→ lots of data needed to densely sample volume

Manifold Hypothesis

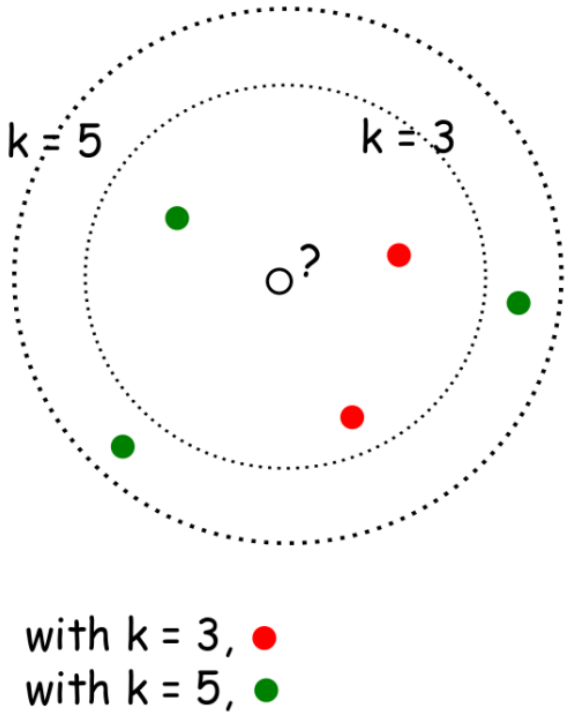
luckily, reality is friendly: most high-dimensional data sets reside on lower-dimensional manifolds → enabling effectiveness of ML





k-Nearest Neighbors (kNN)

- local method, instance-based learning
- non-parametric
- distance defined by metric on $\boldsymbol{x}$ (e.g., Euclidean)



regression:

$$f(\boldsymbol{x}_0) = \frac{1}{k}\sum_{j=1}^{k} y_j$$

with j running over k nearest neighbors of $\boldsymbol{x}_0$

low k: low bias but high variance
high k: low variance but high bias

Image Classification with kNN

choose an image distance, e.g., L1 distance:

$$d(\boldsymbol{I}_1, \boldsymbol{I}_2) = \sum_p |I_1(p) - I_2(p)|$$

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

-

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

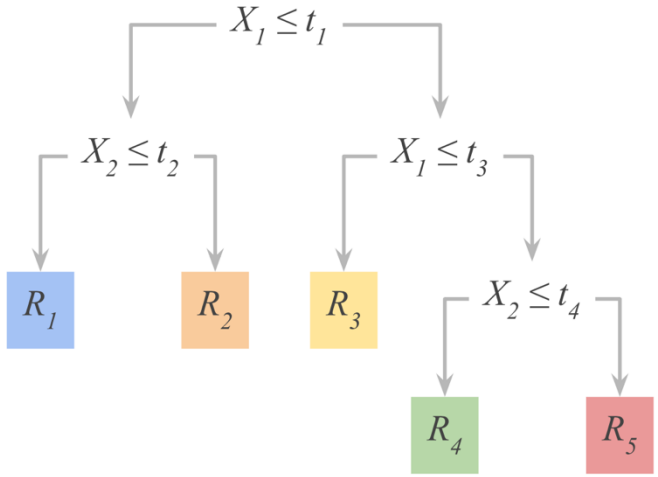
=

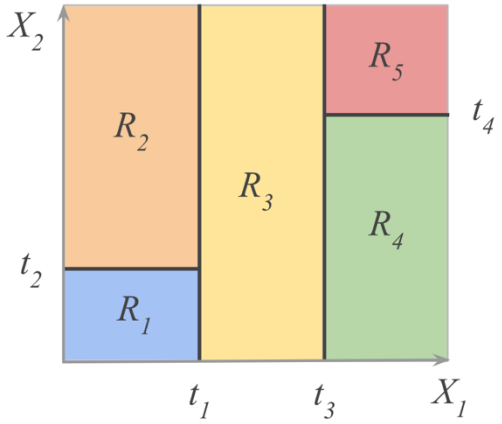
pixel-wise absolute value differences

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

→ 456

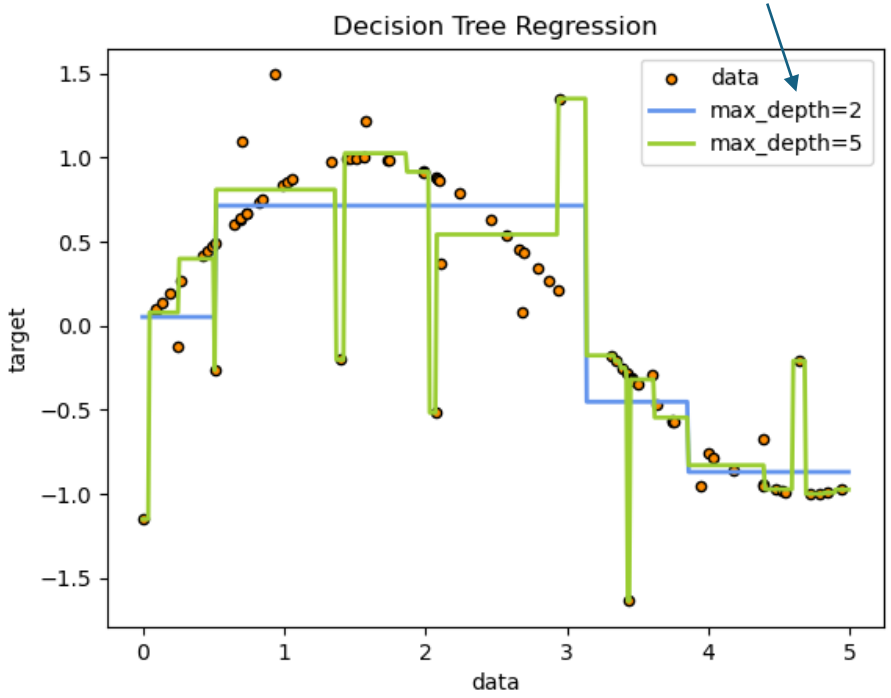
Decision Trees





- non-parametric learning of simple decision rules
- usually, binary trees
- axis-parallel decision boundaries (box-shaped regions in feature space)
- fit constant in each box
  - classification: majority class of target
  - regression: average of target
- fully explainable models

hyperparameter



scikit-learn

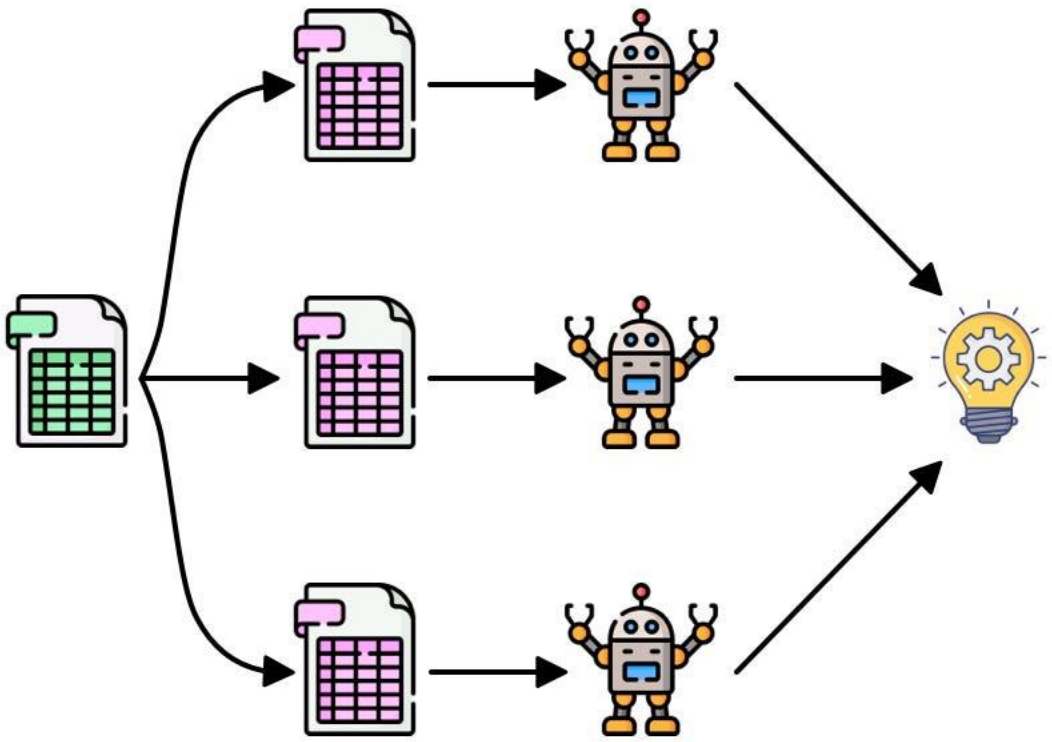
Ensemble Learning

idea: combine several individual models (often of the same type) to form an ensemble model with better predictive performance than each of its constituents

→ learning in several different ways from the training data

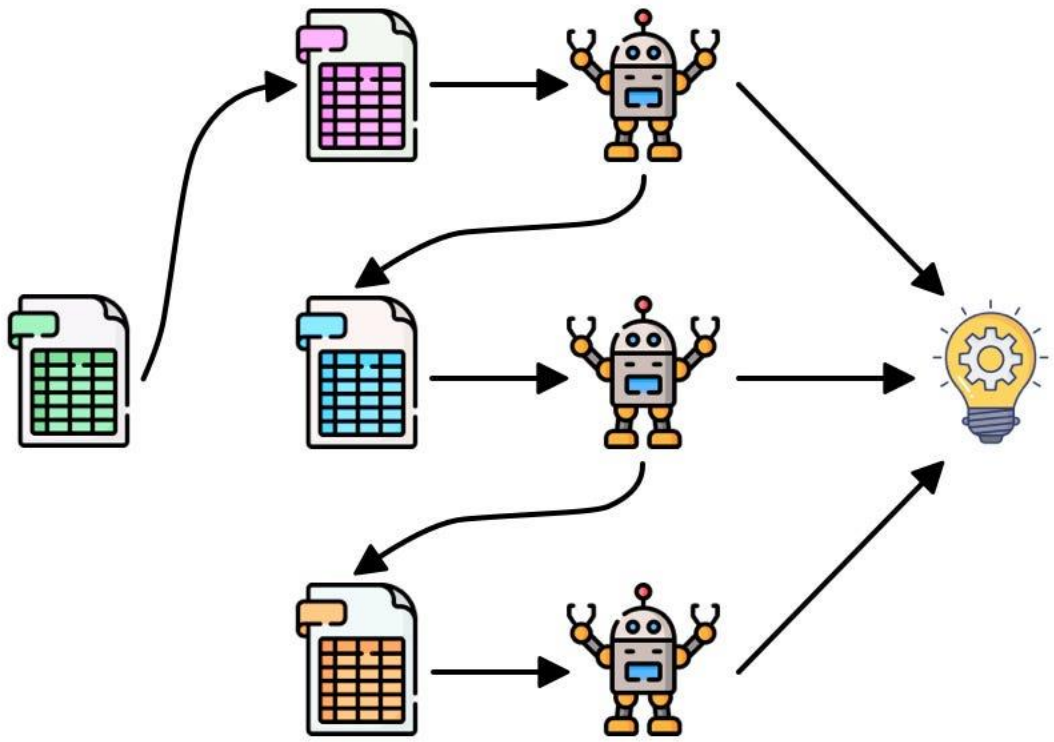
- the trainings of the individual constituent models are (kind of) independent
- outputs of individual models are combined (e.g., averaged)

Bagging



Parallel

Boosting



Sequential

General Recipe of Statistical Learning

ML algorithm by combining:

- **model** (e.g., linear function & Gaussian distribution)
- **objective function** (e.g., squared residuals)
- **optimization algorithm** (e.g., gradient descent)
- **regularization** (e.g., L2, dropout)

Linear Model

data: vector with p different feature values for this example i

$$\boxed{f(\boldsymbol{x}_i) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b}$$

vector with slope parameters (one for each of the p features, but identical for all examples), aka weights

single intercept parameter, aka bias

Loss Function

loss function L: expressing deviation between prediction $\hat{y}$ and target y

$$L(y_i, \hat{y}_i) = L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{w})\big)$$

with $\boldsymbol{w}$ corresponding to parameters of model $f(\boldsymbol{x}_i)$

(for the sake of clarity, include bias b in $\boldsymbol{w}$)

prime example: squared residuals for regression problems

$$L(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$

Cost Function

averaging losses over empirical training data set:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{w})\big)$$

cost function to be minimized according to model parameters $\boldsymbol{w}$

→ objective function

Cost Minimization

minimize training costs $\mathcal{L}(\boldsymbol{w})$ according to model parameters $\boldsymbol{w}$:

$$\nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = 0$$

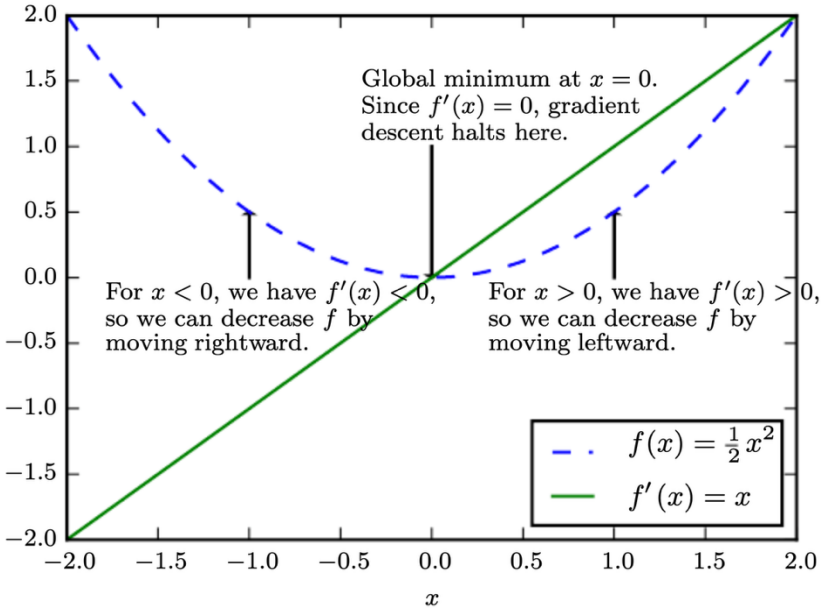
for mean squared error (aka least squares method):

$$\nabla_{\boldsymbol{w}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i; \boldsymbol{w}) \right)^2 = 0$$

Gradient Descent

typically no closed-form solution to minimization of cost function: $\nabla_{\boldsymbol{w}}\mathcal{L} = 0$

→ need for numerical methods



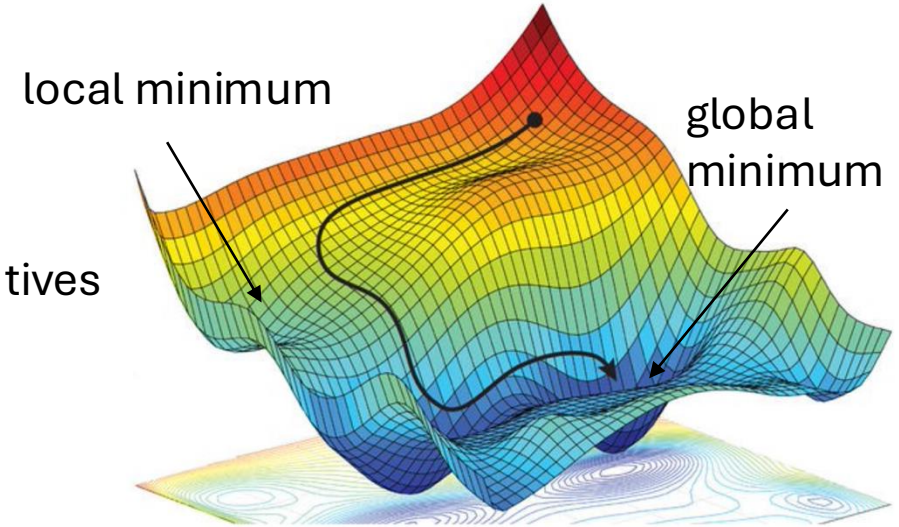
source

popular choice: gradient descent

iteratively moving in direction of steepest descent (negative gradient): $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{w}\mathcal{L}$

step size (learning rate)

vector containing all partial derivatives

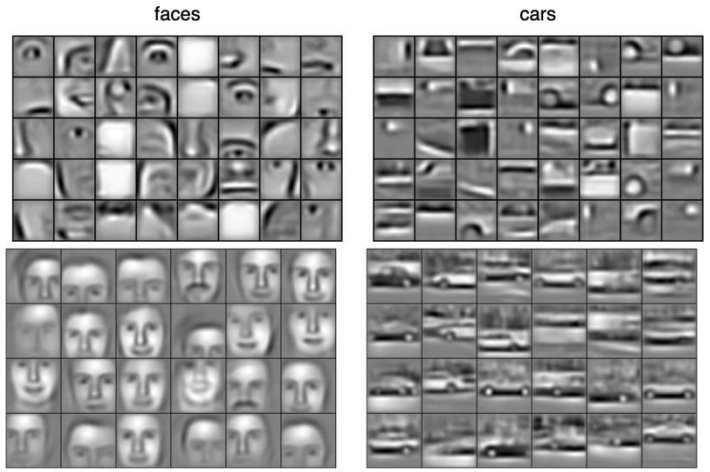


Ladder of Generalization

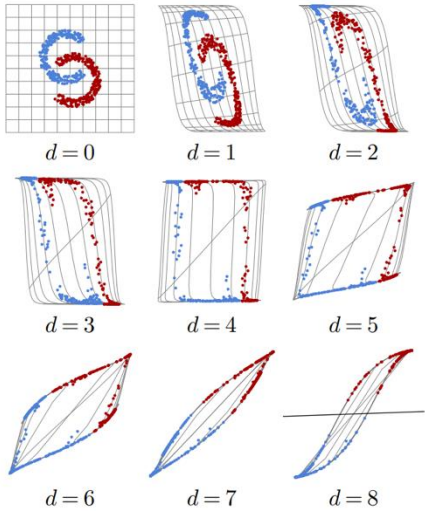
classic ML: feature engineering

deep learning: feature learning

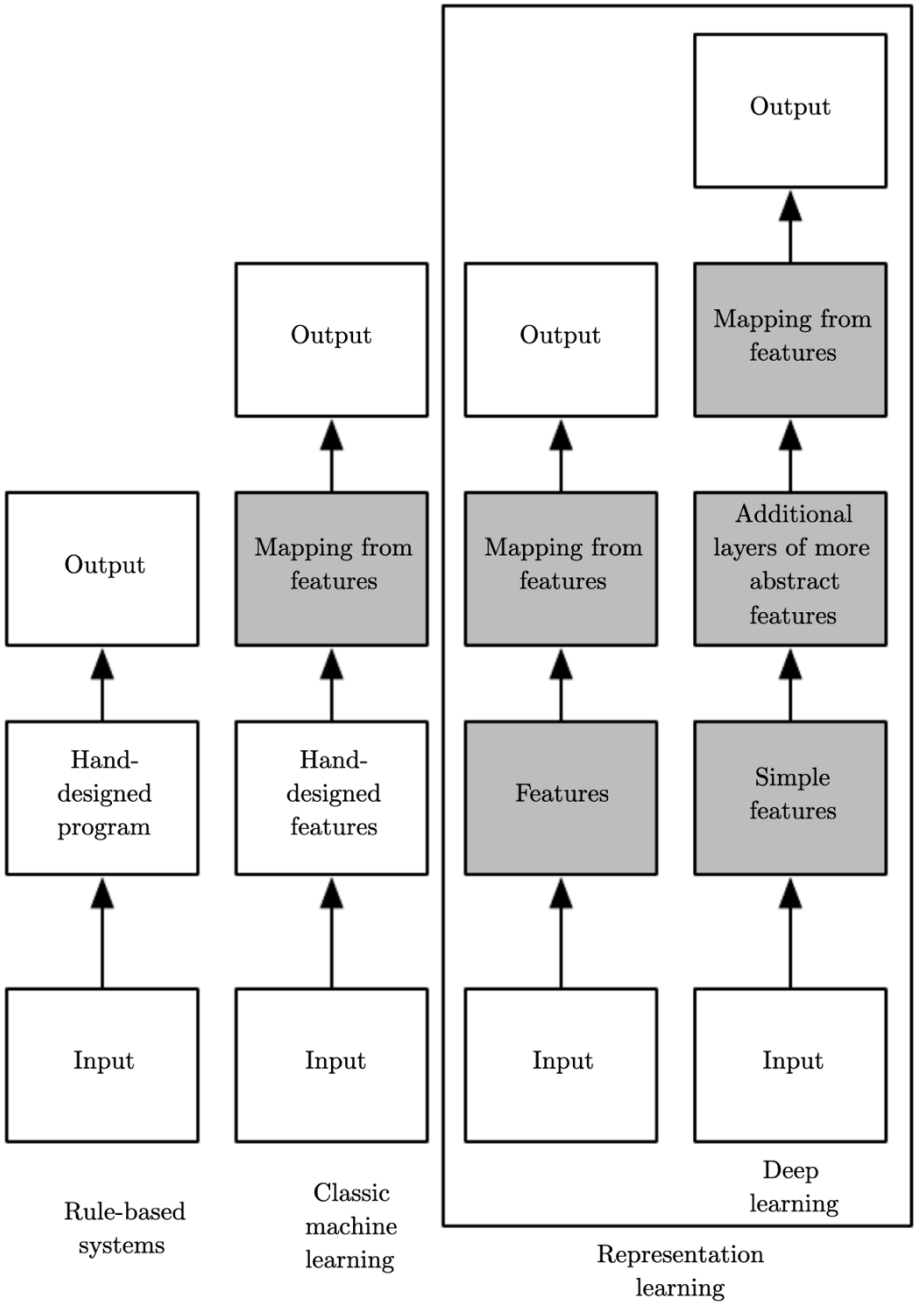
(hierarchy of concepts learned from raw data in deep graph with many layers)



source



source



source

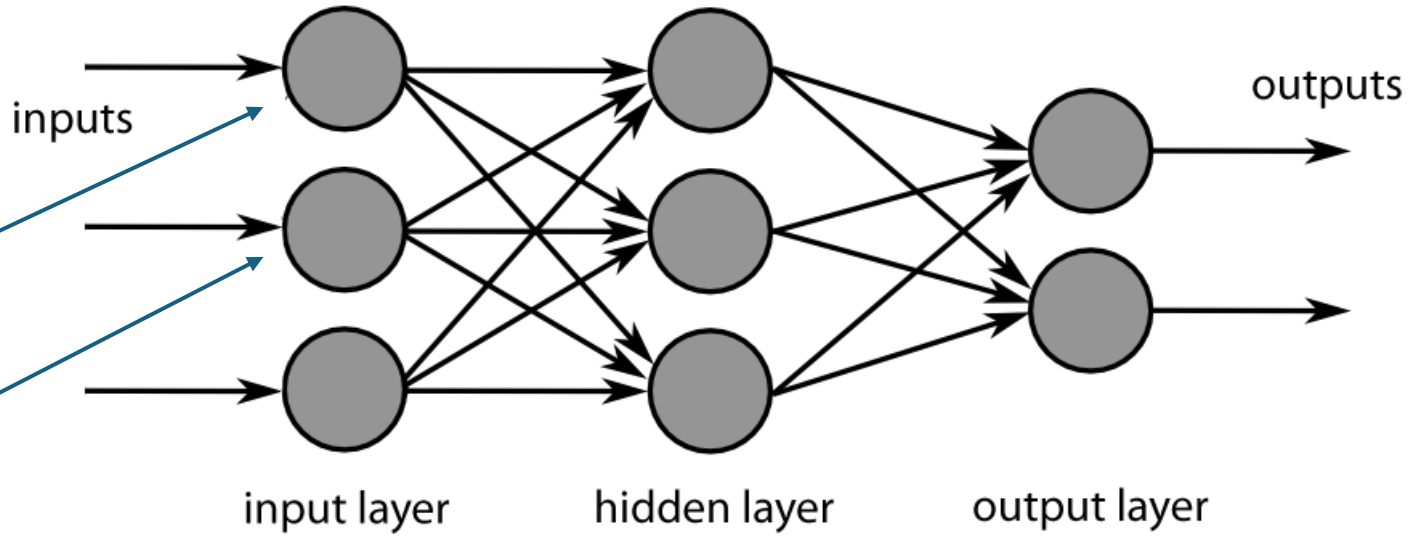
Neural Networks

idea: powerful algorithm by combining many linear building blocks

e.g., digital image

intensity pixel 1

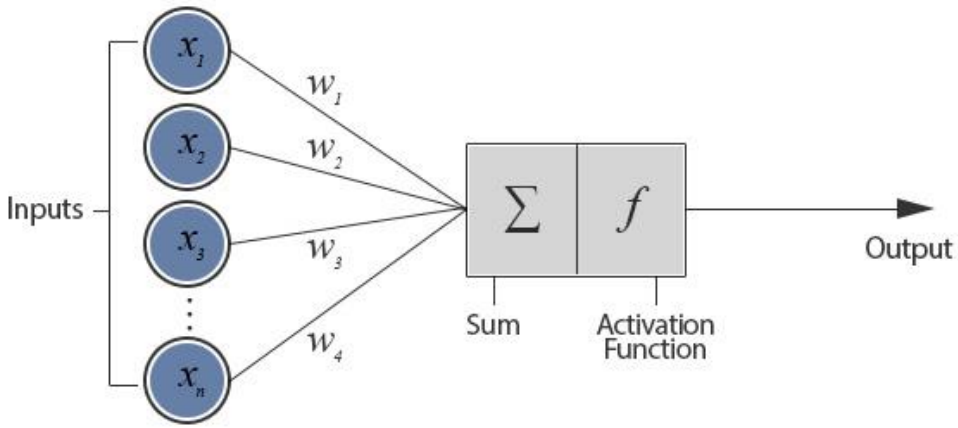
intensity pixel 2



each node exchanges information only with directly connected nodes
→ parallel computation

Artificial Neuron

artificial neuron (node in neural network):

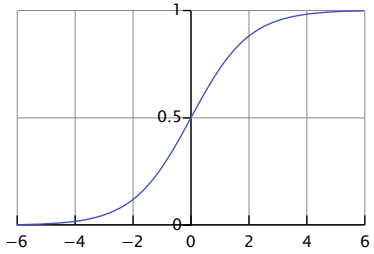


linear model with parameters called weights $\boldsymbol{w}$ (including bias, not shown for simplicity)

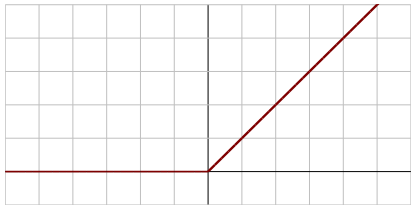
non-linear via (differentiable) activation function on top of linear model

examples:

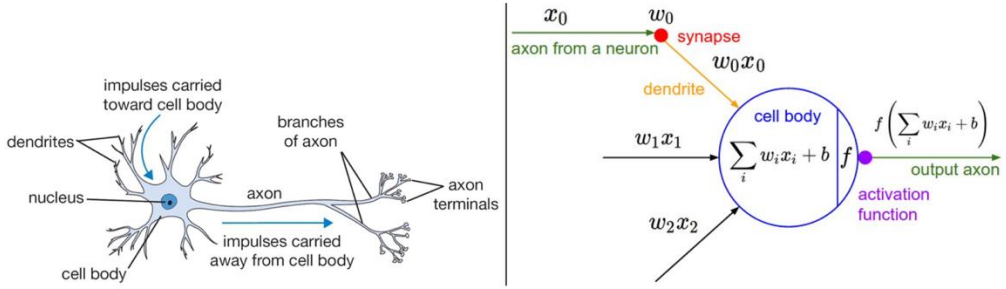
- sigmoid



- ReLU (Rectified Linear Unit)



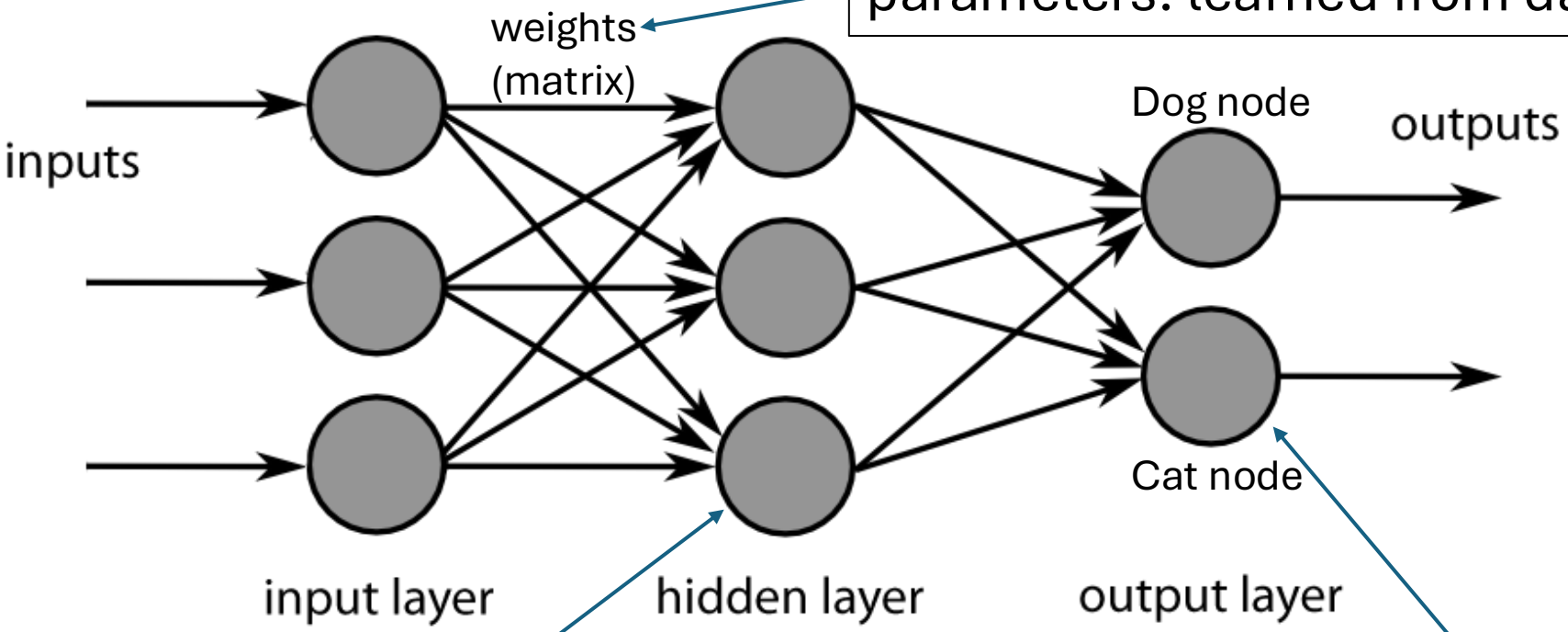
inspired from biological neurons:



source

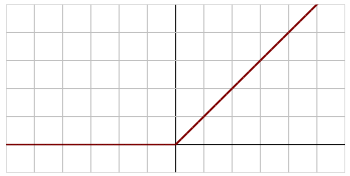
Neural Network as Nonlinear Function

parameters: learned from data



e.g., node 3

linear transformation with nonlinear activation functions



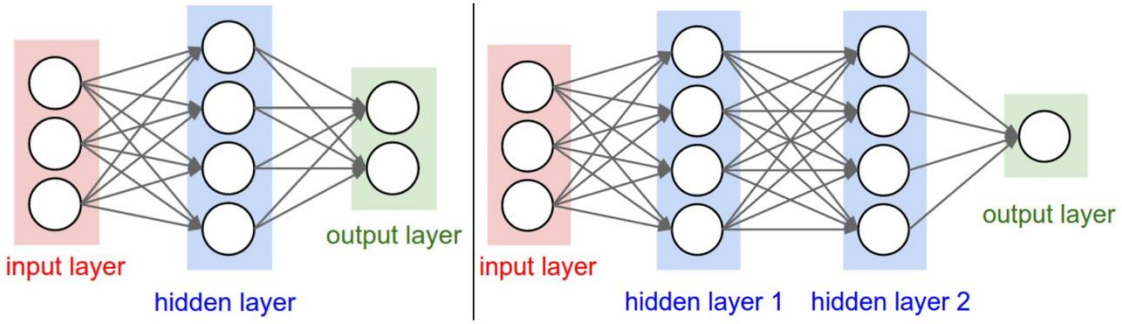
$$\left(\overrightarrow{w_3} \cdot \vec{x} \right)$$

probability for „cat“
(softmax instead of activation function)

Multi-Layer Perceptron (MLP)

fully-connected feed-forward network with at least one hidden layer (universal function approximator)

binary classification



regression

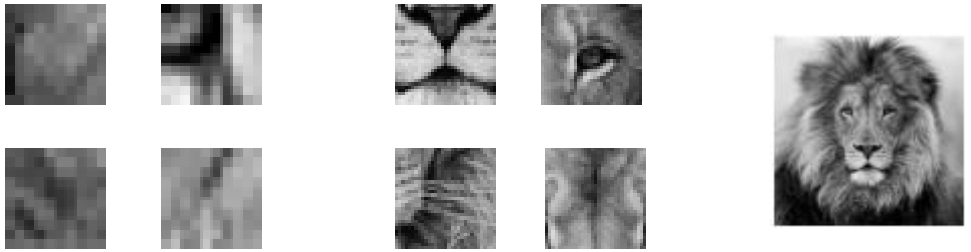
toward deep learning: add hidden layers

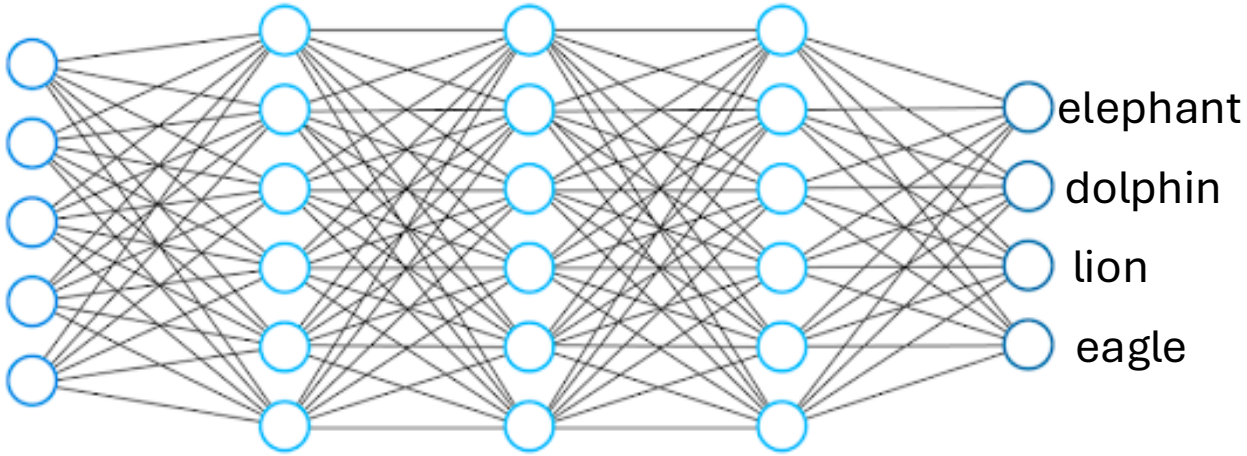
more layers (depth) more efficient than just more nodes (width): less parameters needed for same function complexity

Hierarchical Representation

deep learning:
several hidden layers

increasing abstraction





Classification: Logistic Regression

binary classification: $y = 1$ or $y = 0$

→ predict probabilities p_i for $y = 1$

- logit (log-odds) as link function
- Y following Bernoulli distribution

$$\mathrm{logit}(E[Y|\boldsymbol{X} = \boldsymbol{x}_i]) = \ln\left(\frac{p_i}{1 - p_i}\right) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b$$

Multi-Classification: Softmax

for classification in K categories:

- use “score” function with K outputs

$$f_k(\boldsymbol{x}_i) = \boldsymbol{w}_k \cdot \boldsymbol{x}_i + b_k$$

- and convert into probabilities by means of softmax function

$$\frac{\exp\left(f_k(\boldsymbol{x}_i)\right)}{\sum_{l=1}^{K} \exp\left(f_l(\boldsymbol{x}_i)\right)}$$

Classification Networks

cross-entropy loss (one output node for each class k):

$$L(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

using softmax on output nodes:

$$\hat{y}_k = \frac{e^{o_k}}{\sum_{l=1}^{K} e^{o_l}}$$

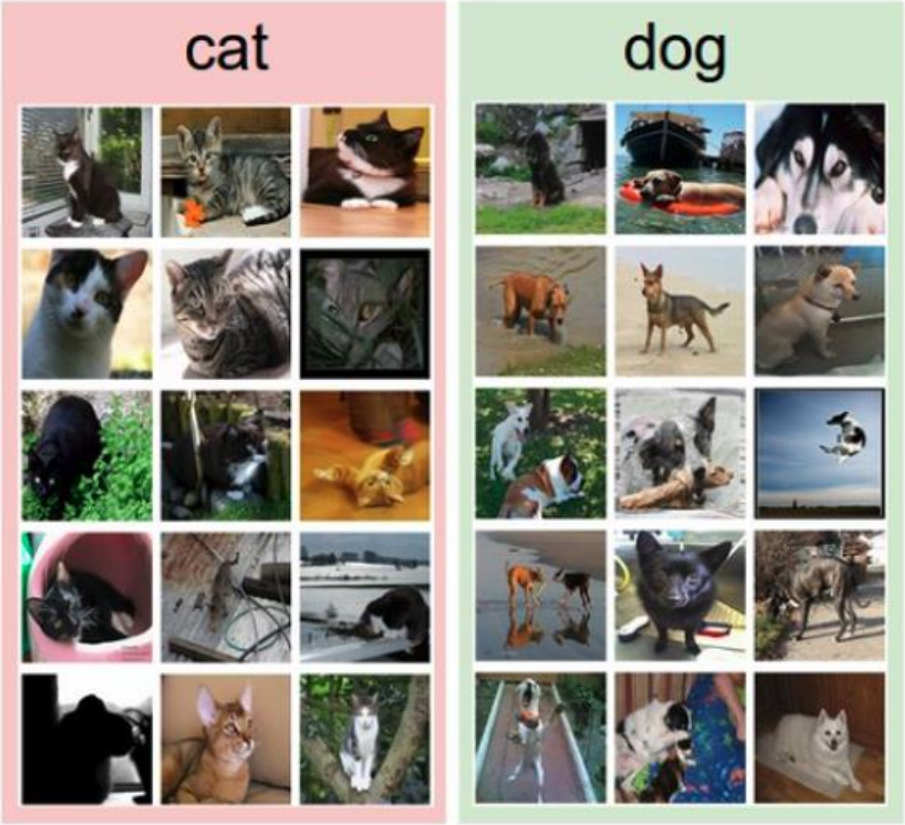
→ prediction of probabilities

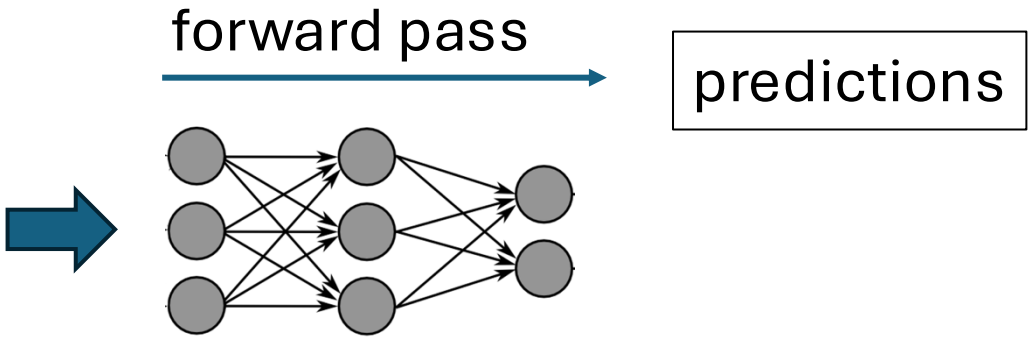
regression networks:

- squared error loss: $L(y, \hat{y}) = (y - \hat{y})^2$
- just one output node
- no function on output → prediction of real numbers

Training with Labeled Images

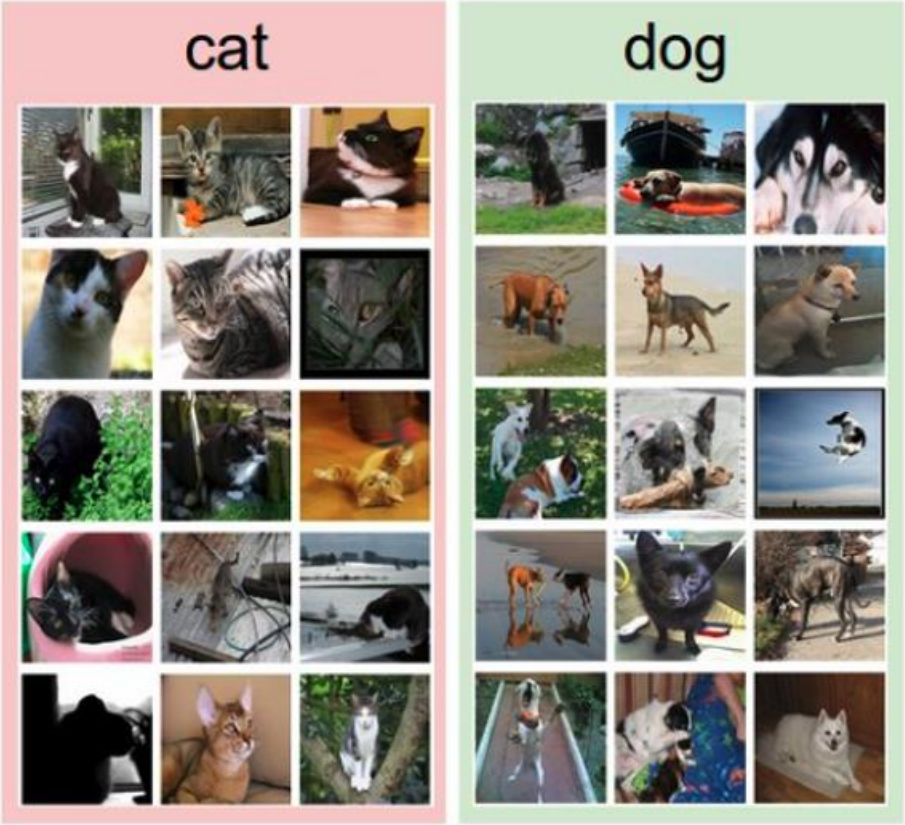
label:

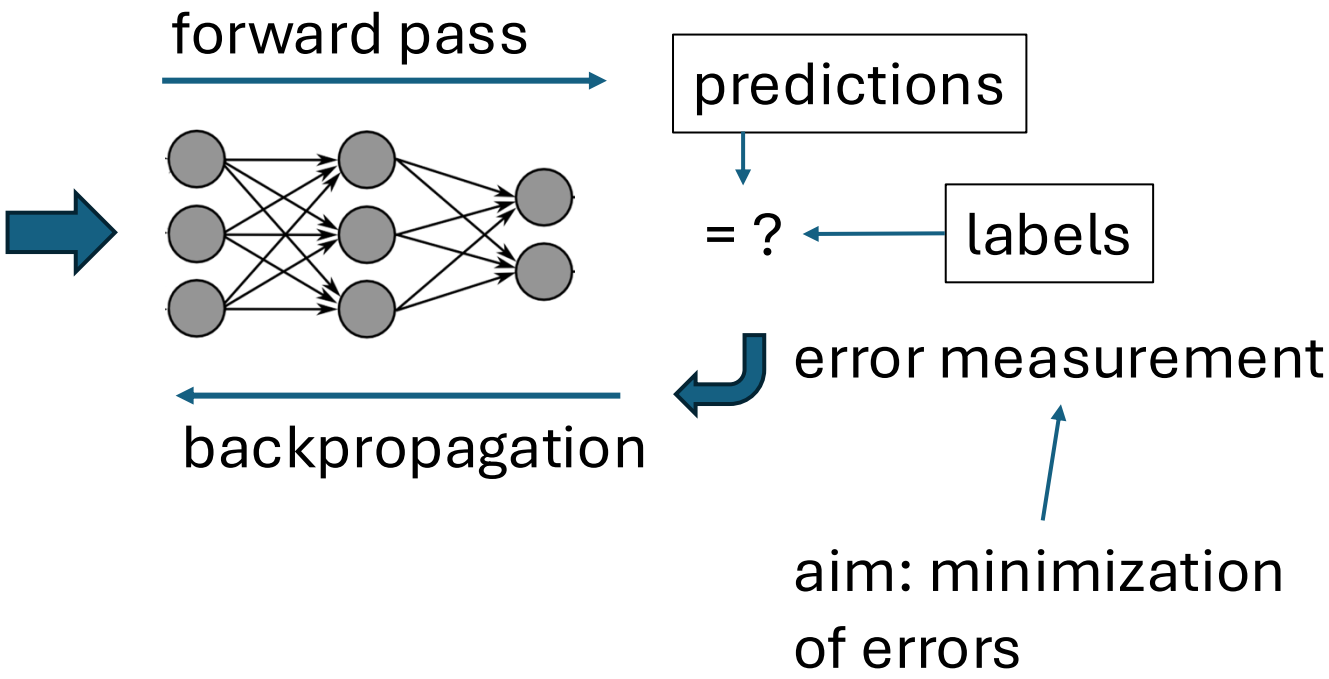




Training with Labeled Images

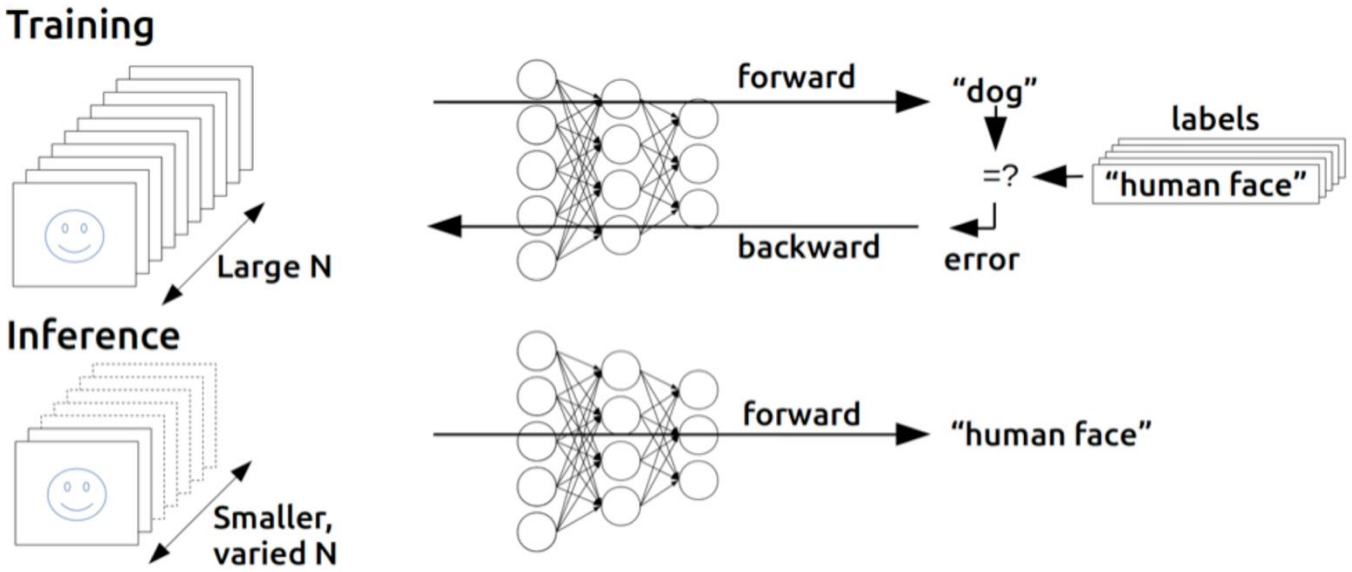
label:





Find Gradients for (Deep) Neural Networks

backpropagation of errors through network layers via chain rule of calculus (enables learning of deep neural networks)



source

forward pass:

- fix current weights
- compute predictions

backward pass:

- compute errors
- calculate gradients (backpropagation of errors)
- update weights accordingly (gradient descent)

Training: Iterative Adjustment of Weights

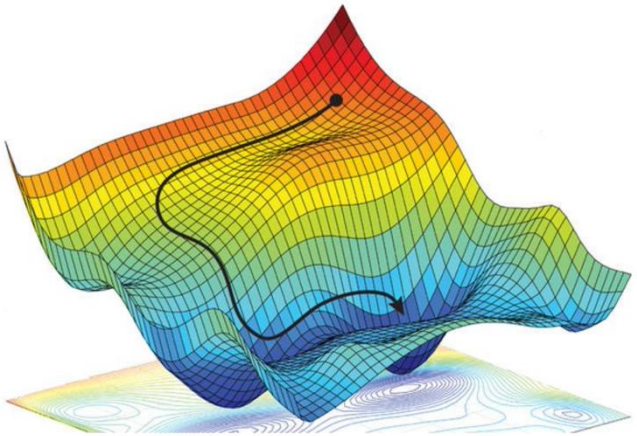
```
for epoch in range(epochs):
    for X, y in mini_batches:
        pred = model(X)
        cost = loss(pred, y)
        cost.backward()
        optimizer.step()
```

quantification of error (here: cross entropy)

chain rule to calculate gradient (direction) of cost function with respect to weights

one step in direction of steepest descent: (stochastic) gradient descent

Stochastic Gradient Descent (SGD)



step size cost function

weight updates: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} \mathcal{L}$

all training examples

options:

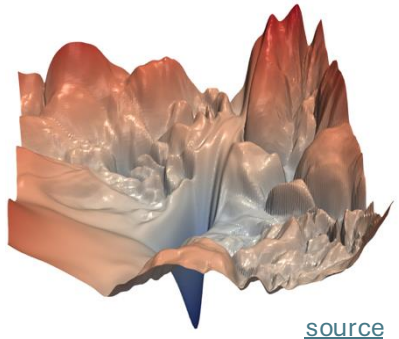
- $\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} L_i$ batch gradient descent (whole training dataset)
- $\mathcal{L} = L_i$ stochastic gradient descent (single example)
- $\mathcal{L} = \frac{1}{m} \sum_{i=1}^{m} L_i$ mini-batch stochastic gradient descent

mini-batch size → hyperparamter: tradeoffs between runtime & memory, convergence & generalization

random fluctuations of SGD help to escape saddle points

How to Train Deep Neural Networks Effectively?

typical loss surface:



source

optimization and regularization difficult

- local vs global minima, saddle points, easily overfitting
- many hyperparameters to tune

but many methods to get it working in practice (despite partly patchy theoretical understanding)

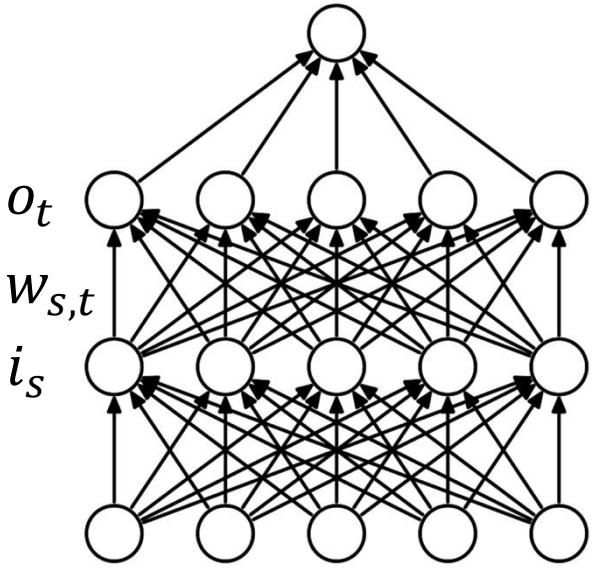
- optimization: activation and loss functions, weight initialization, adaptive learning rate (e.g., Adam), learning rate schedulers
- explicit regularization: weight decay, dropout, data augmentation
- implicit regularization: early stopping, batch/layer normalization, SGD

Feed-Forward Neural Networks

computation in usual feed-forward network:

matrix multiplication of scalar inputs and weights

$$o_t = \sum_s w_{s,t}\, i_s$$



dropped dimension of different training observations (in mini-batch) in this view

Inductive Bias

plain feed-forward networks very inefficient for image classification:

make no use of spatial structure (locality of objects)

→ need to learn it from scratch

better way: introduce it right in the architecture of the ML method (called inductive bias)

→ convolution with spatial filters (**learned** rather than handcrafted)

Grid-Like Data

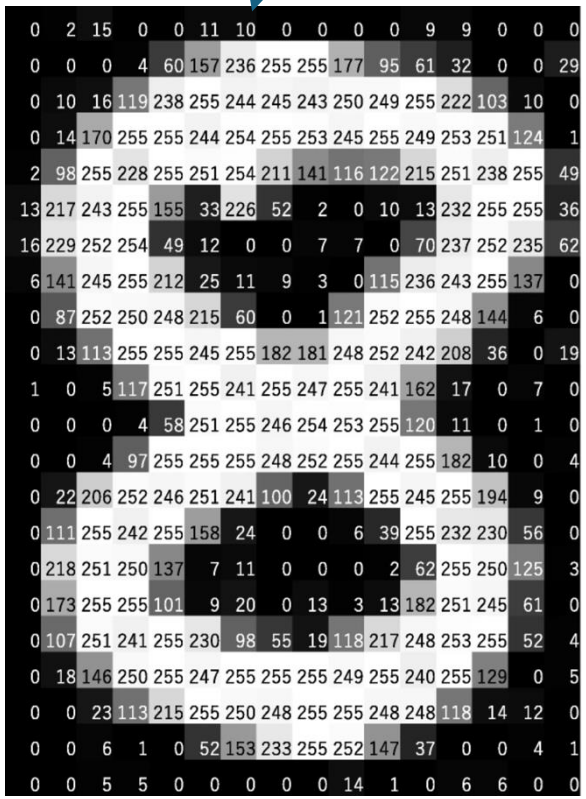
image data: 2-D grid of pixels (spatial structure)

convolutional networks:

neural networks using learned convolution kernels as weights (in place of general matrix multiplications)

→ highly regularized feed-forward networks

scalar values (like in usual feed-forward network)



source

Convolution Operation

actually, correlation (convolution would have – here):

$$o_{x,y} = \sum_{s,t} w_{s,t}\, i_{x+s,y+t}$$

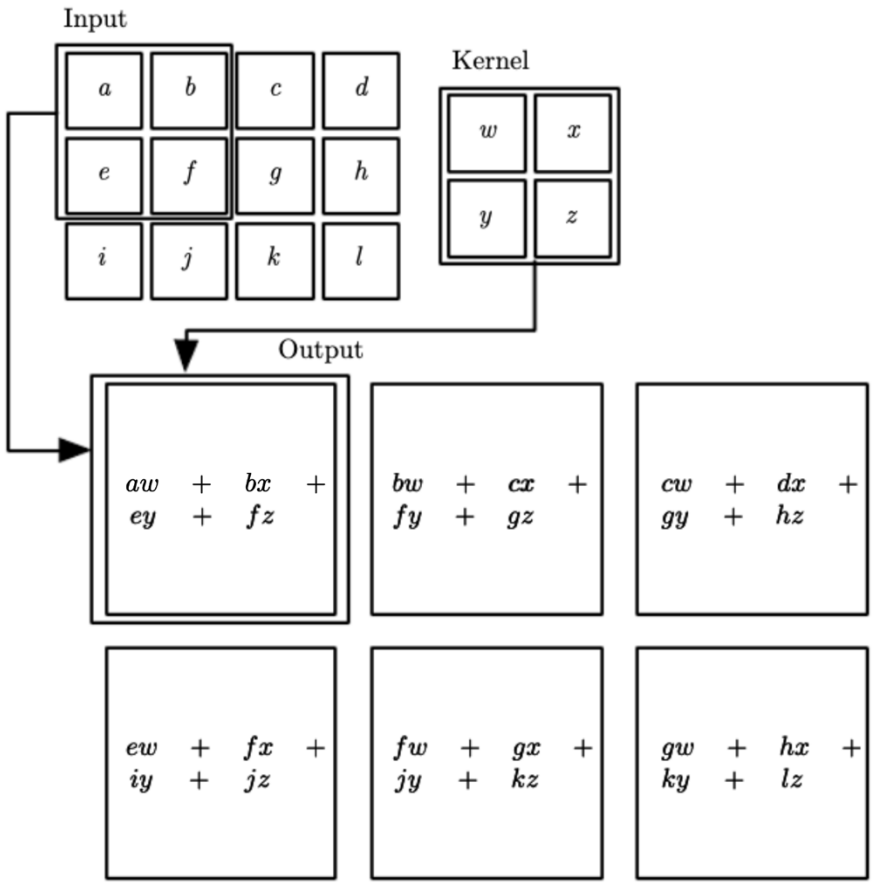
feature map
(transformed image)

kernel
(learned)

image
(input or feature map)

(again, dropping dimension of different training observations)

2D convolution

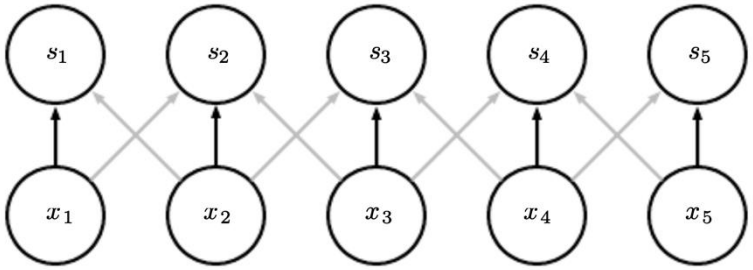


source

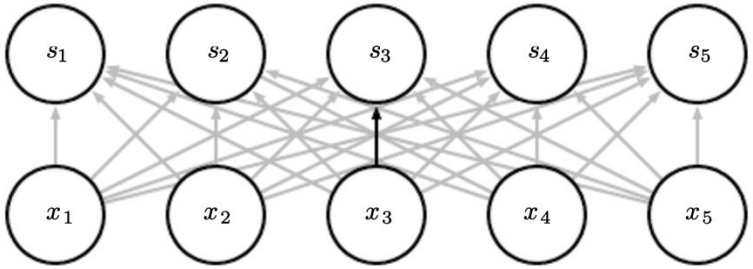
Regularization Effects

- sparse interactions: much less weights
- parameter sharing: use same weights for different connections

convolutional layer:



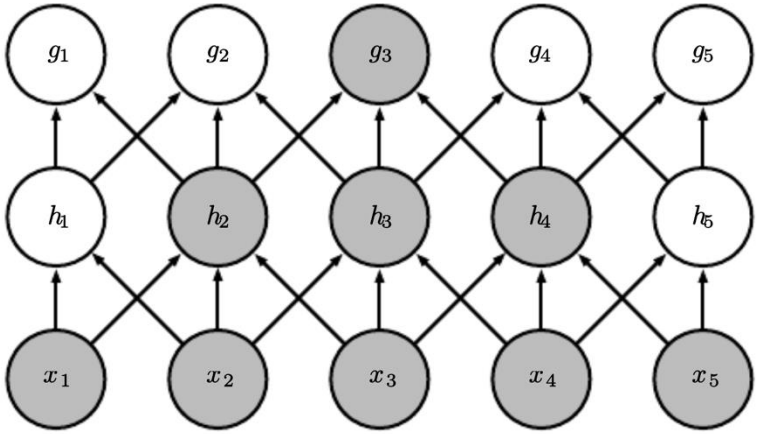
fully-connected layer:



effect of receptive field over several layers:

- consider only locally restricted number of input values from previous layer
- grows for earlier layers (indirect interactions)

→ hierarchical patterns from simple building blocks (many aspects of nature hierarchical)

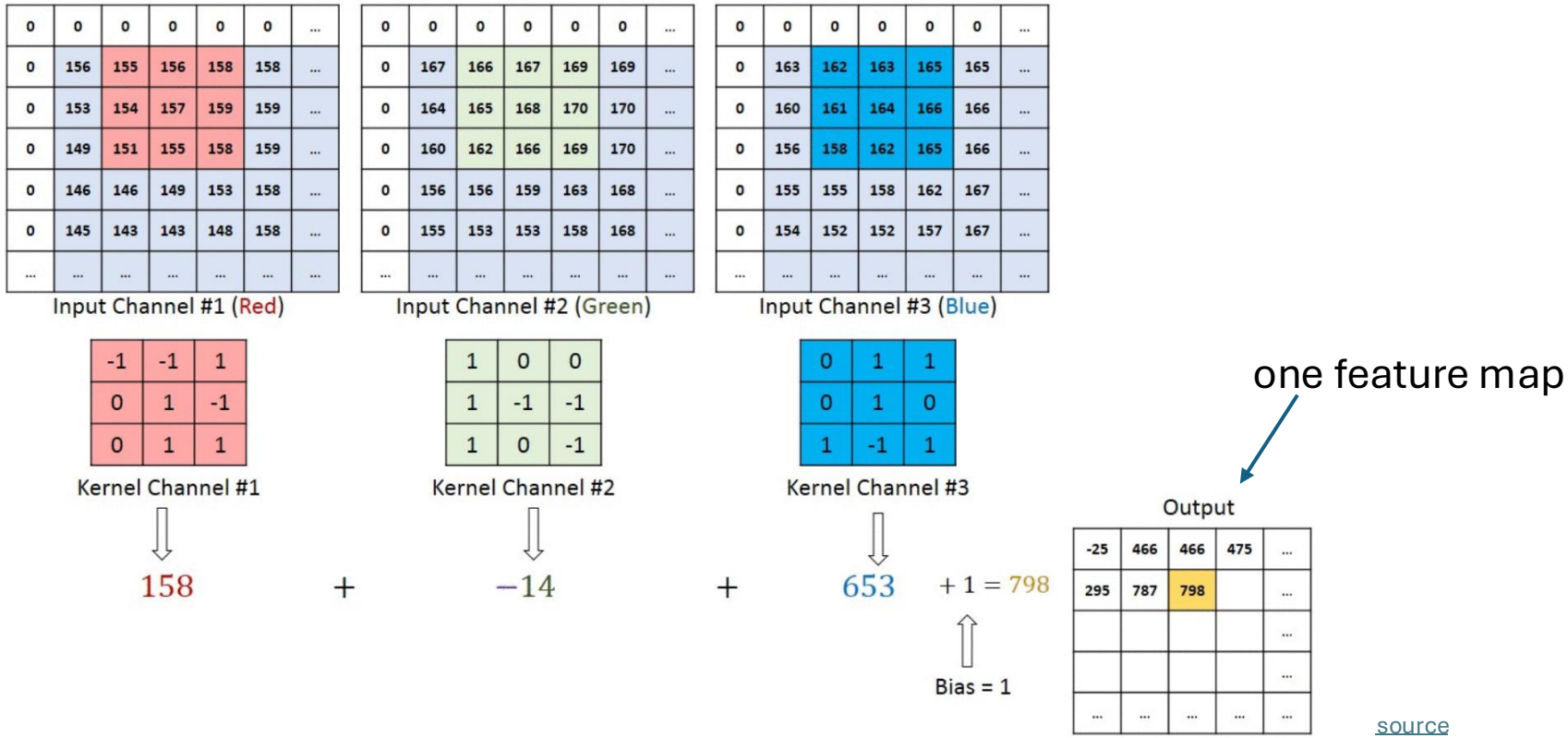


source

Channel Mixing

several input channels c (RGB) and several output channels f (different feature maps):

$$o_{f,x,y} = \sum_{c,s,t} w_{f,c,s,t}\, i_{c,x+s,y+t}$$



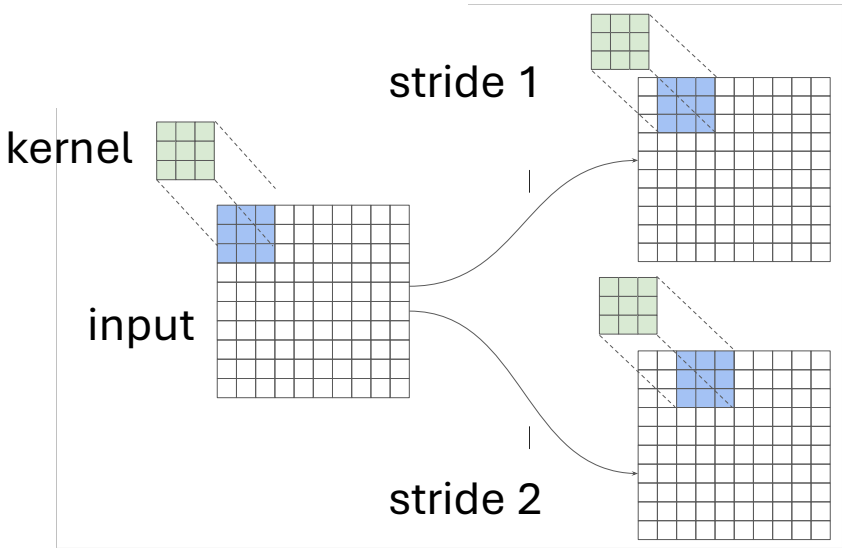
source

Striding and Padding

need to define how to stride over image

stride > 1 corresponds to down-sampling

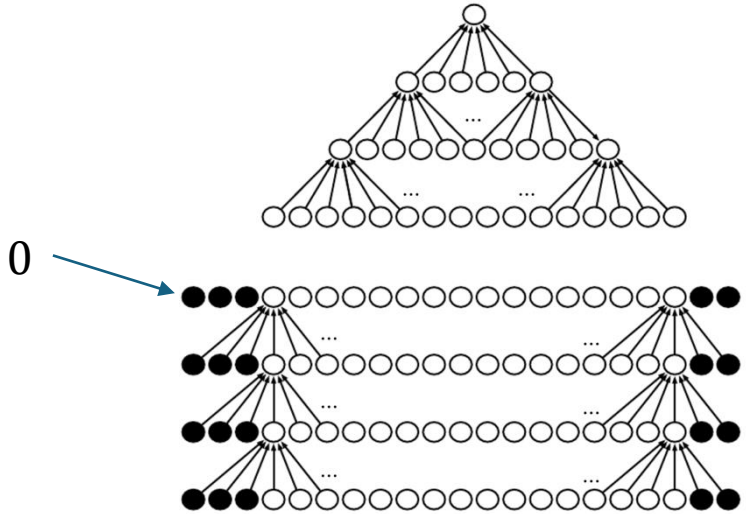
→ fewer nodes after convolutional layer



zero-padding of input to make it wider:

otherwise shrinking of represenation with each layer (depending on kernel size)

→ needed for large kernels and several layers



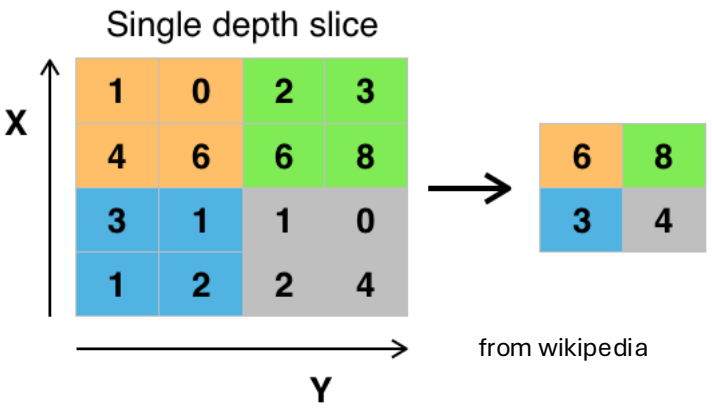
source

Another Ingredient: Pooling

replacing outputs of neighboring nodes with summary statistic (e.g., maximum or average value of nodes)

→ non-linear downsampling (regularization)

max pooling:

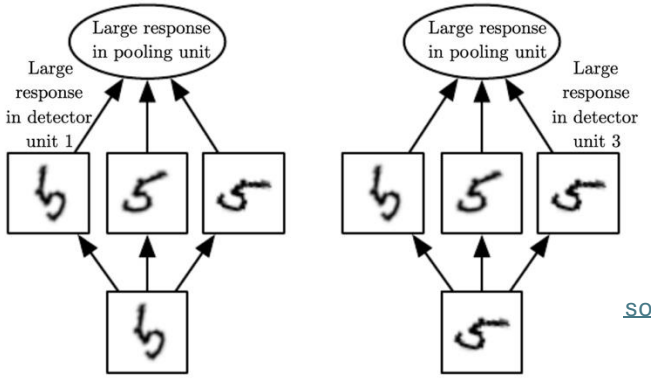


from wikipedia

pooling is translation invariant: no interest in exact position of, e.g., maximum value

pooling over features learned by separate kernels (cross-channel pooling) can also learn other transformation invariances, like rotation or scale

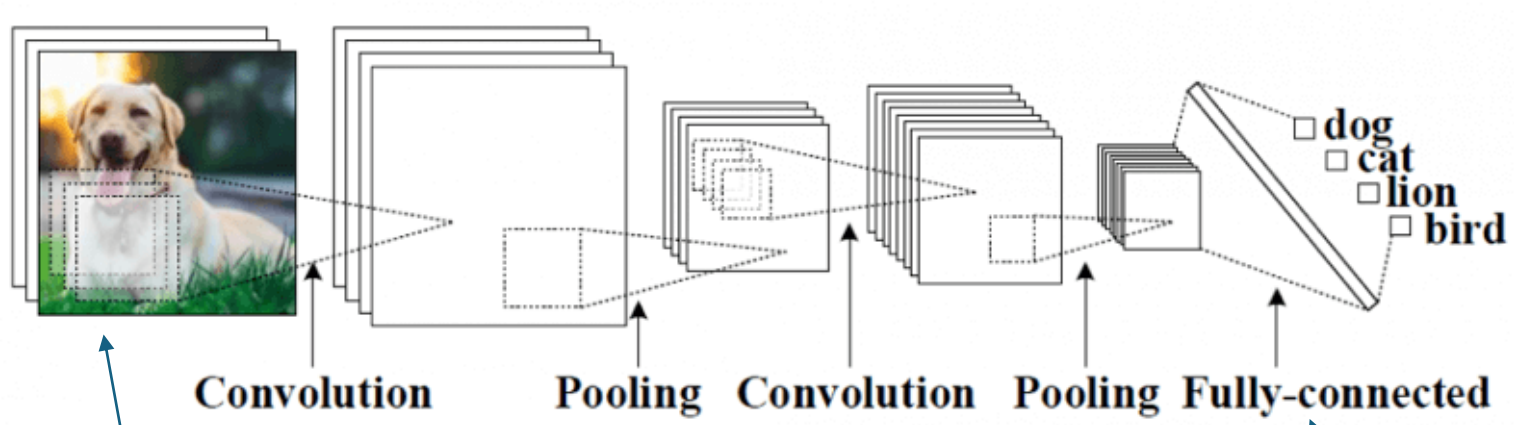
(convolutions can detect same translated motif across entire image, but not rotated or scaled versions of it)



source

Putting It All Together

convolutional neural networks in short:
local connections, shared weights, pooling, many layers

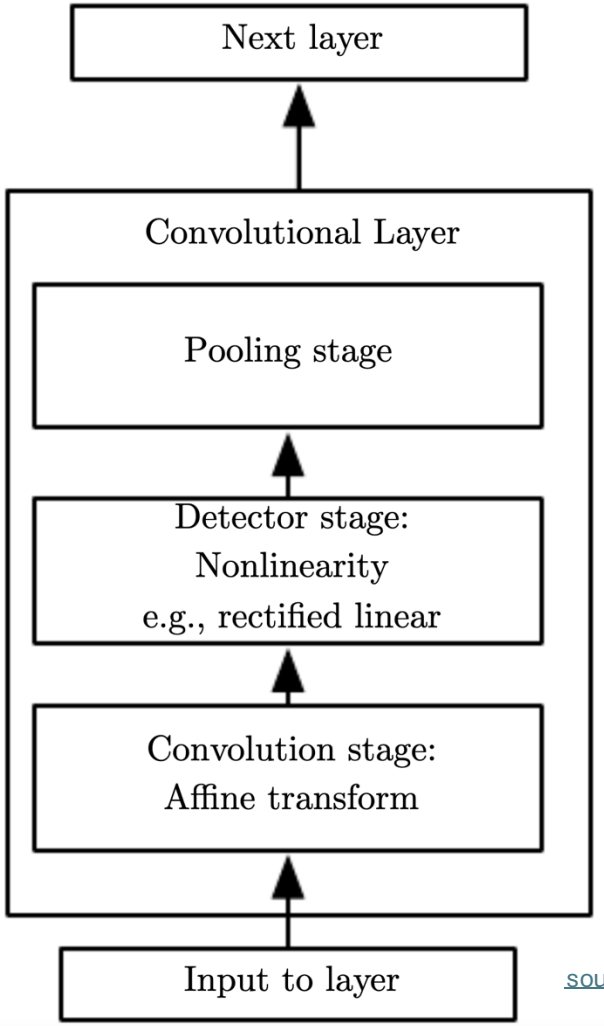


many images (training examples), potentially with several channels

several kernels producing several feature maps

downsampling

flatten dimensions for final classification



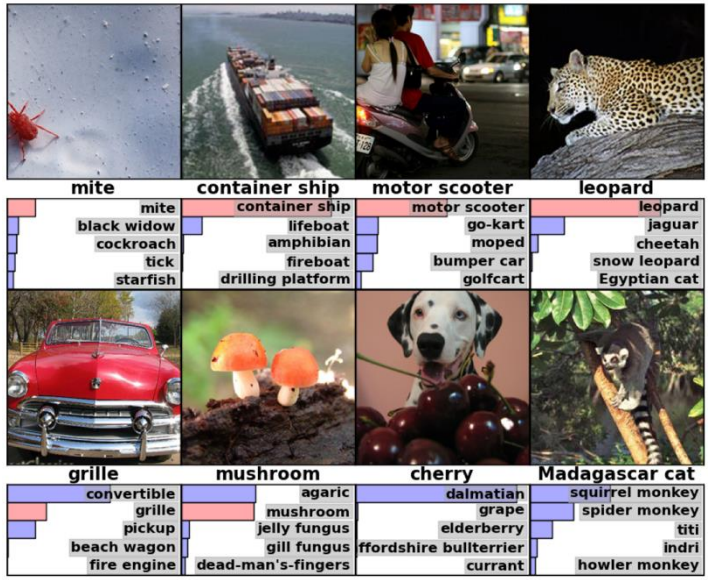
source

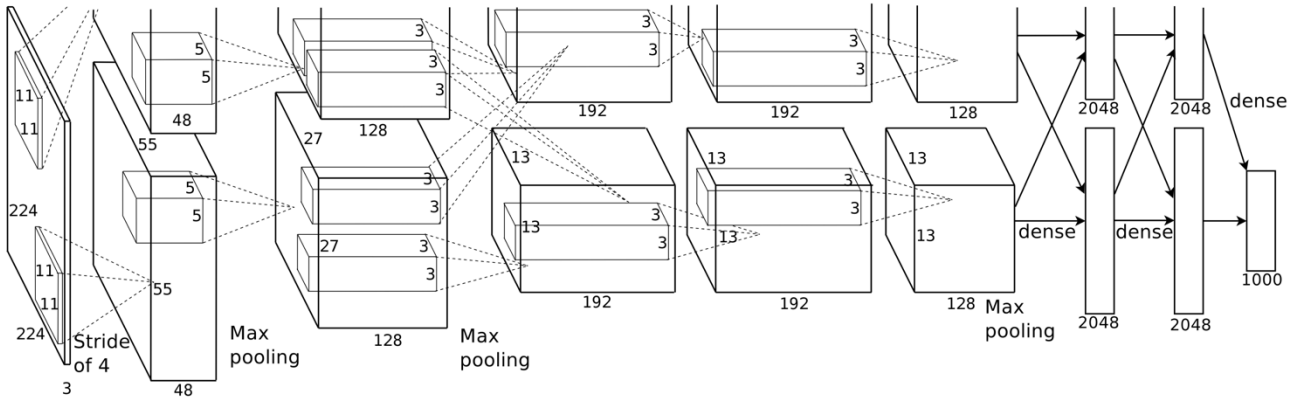
Rise of Deep Learning

a little bit oversimplified:

deep learning = lots of training data + parallel computation + smart algorithms

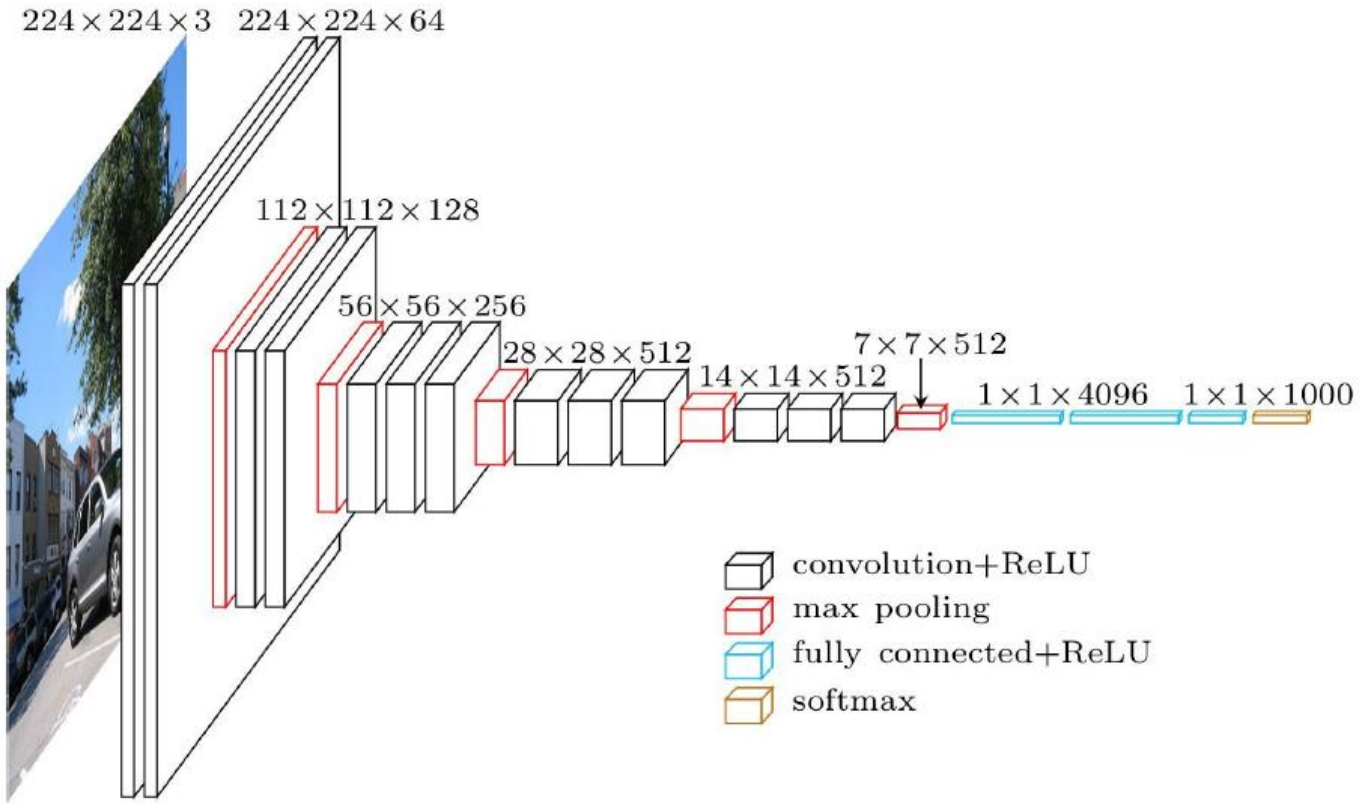
AlexNet: ImageNet (with data augmentation) + GPUs + ReLU, dropout, SGD

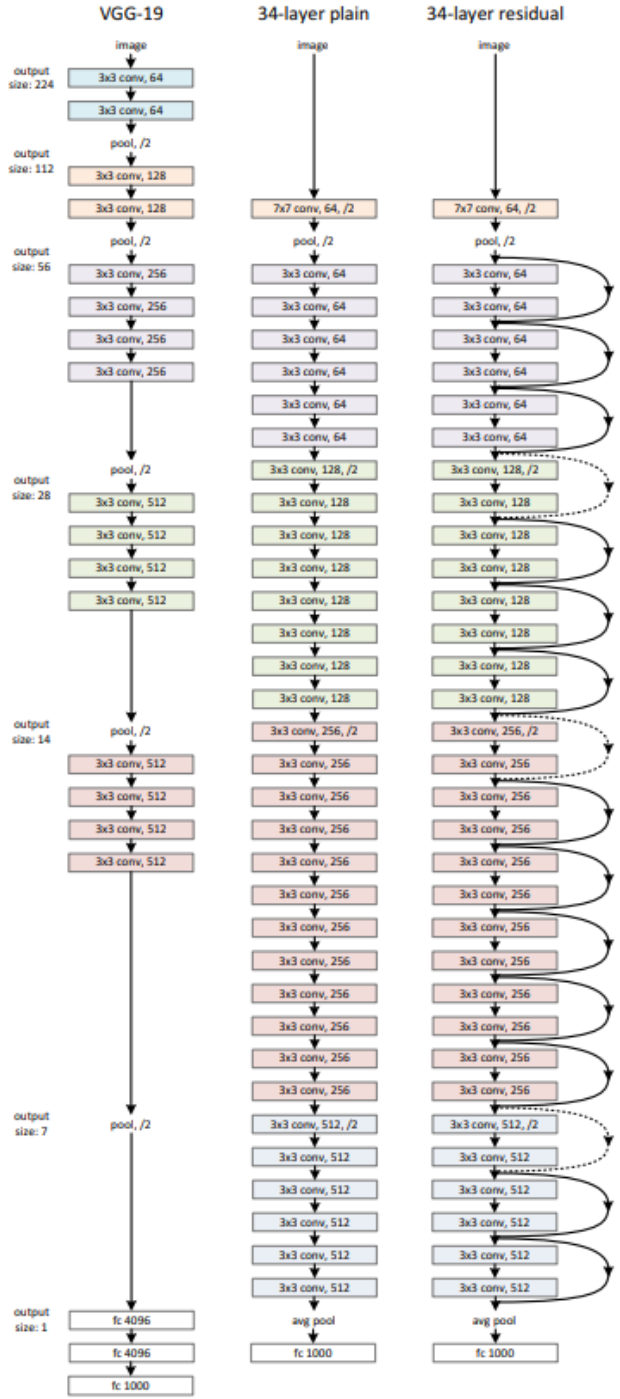




source

VGG


224 × 224 × 3
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
1 × 1 × 4096
1 × 1 × 1000
convolution+ReLU
max pooling
fully connected+ReLU
softmax


VGG-19
34-layer plain
34-layer residual
image
3x3 conv, 64
pool, /2
3x3 conv, 128
3x3 conv, 256
3x3 conv, 512
fc 4096
fc 1000
7x7 conv, 64, /2
3x3 conv, 128, /2
3x3 conv, 256, /2
3x3 conv, 512, /2
avg pool

ResNet